Autonomous Agent Systems for Discovery and Engineering

by

Xiaoyuan Liu
120040051

CSC6032 Final Report

School of Data Science
The Chinese University of Hong Kong, Shenzhen
May 2026

Abstract

Autonomous agent systems are emerging as practical mechanisms for scientific discovery and engineering by augmenting large language models with tool use, program execution, memory, and evaluator feedback. Rather than prompting a model to generate a final answer in a single pass, these systems instantiate iterative workflows in which candidate programs are proposed, executed, scored, repaired, and then used as evidence for subsequent search. This report examines three representative papers that illustrate a recent progression in this direction: AlphaEvolve, EvoX, and CORAL. Respectively, these works exemplify evolutionary code improvement within a fixed search scaffold, meta-evolution of the search strategy itself, and autonomous multi-agent evolution supported by shared persistent memory.

The central argument of this report is that the literature is advancing along three connected axes: the degree of agent autonomy, the adaptivity of the search process, and the reservoir mechanism through which knowledge accumulates across attempts. AlphaEvolve demonstrates that frontier LLMs can function as reliable discovery engines when their proposals are grounded in execution-based evaluation and organized through a program database. EvoX shows that the search strategy should not remain fixed, since effective exploration and exploitation depend on both the current population state and the stage of the run. CORAL extends this trajectory by delegating the retrieve-propose-evaluate-update loop to long-running multi-agent systems that coordinate through attempts, notes, and skills stored in shared memory. Using a unified comparative framework, this report analyzes each paper's motivation, system design, evaluation protocol, results, strengths, limitations, and role in the broader movement toward autonomous discovery systems. It concludes that these systems are most promising in domains with strong automated evaluators, while reliable deployment still requires careful control over evaluation validity, computational cost, reproducibility, memory quality, and human oversight.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Scientific discovery and engineering optimization are often described as creative activities, but a large part of the day-to-day process is procedural. A researcher forms a hypothesis, writes code, runs an experiment, reads the traces, keeps the useful part, and tries again. A systems engineer follows a similar loop when improving a scheduler, a kernel, or a heuristic: propose a change, measure it under a workload, diagnose the failure mode, and refine the program. Large language models are useful in this setting because they can write and explain code, but raw generation is not enough. A model can invent plausible algorithms that fail silently, overfit to examples, or ignore constraints. The central question is therefore not only whether an LLM can propose ideas, but whether an agentic system can organize repeated proposal, execution, feedback, and knowledge reuse well enough to make genuine progress.

The three papers reviewed in this report answer this question at different levels of autonomy. AlphaEvolve presents an evolutionary coding agent that improves whole programs under automatic evaluation and stores previous attempts in a program database. EvoX observes that such systems still rely on fixed search rules and introduces an outer loop that evolves the search strategy itself. CORAL treats the remaining scaffolding as another source of rigidity and lets long-running agents decide what to inspect, when to evaluate, and what knowledge to externalize, while coordinating through shared persistent memory. Taken together, these papers form a coherent story: discovery systems are shifting from LLMs as mutation operators inside fixed loops toward autonomous agents that control more of the search process.

This report is written as a comparative literature review rather than as an implementation paper. Its goal is to explain how the three systems work, why their design choices matter, and where the evidence supports broader conclusions. The comparison is especially important because the papers differ not only in algorithmic details but also in their assumptions about tasks. AlphaEvolve is strongest when high-throughput evaluation is available and the candidate solution can be represented as code. EvoX targets the same general family of evaluator-guided optimization

problems but focuses on strategy adaptation under a fixed budget. CORAL targets open-ended tasks where long-horizon knowledge accumulation and multi-agent exploration can matter as much as any single mutation operator.

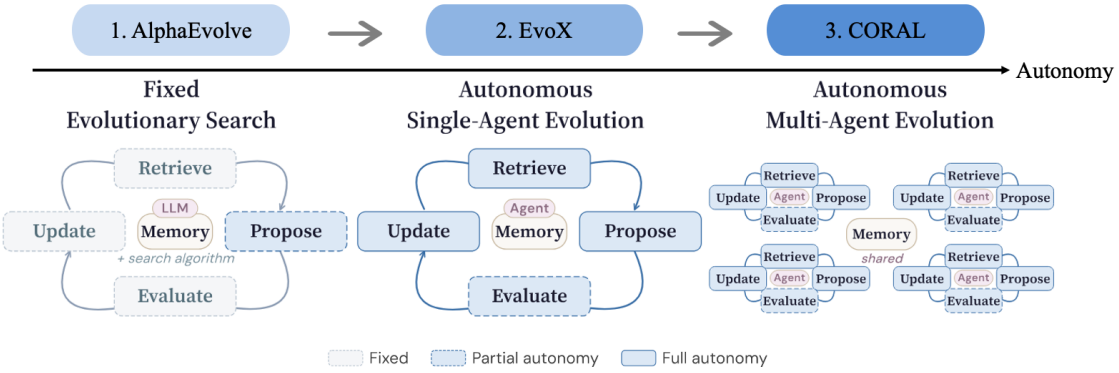


Figure 1.1: Conceptual roadmap of the three reviewed systems along the autonomy axis.

1.1 Research Target

The common target in the three papers is not ordinary answer generation. It is iterative discovery under feedback. A one-shot LLM call receives a prompt and returns a candidate answer; if that answer is wrong or mediocre, the call itself has no durable mechanism for learning from the failure. In contrast, AlphaEvolve, EvoX, and CORAL all turn generation into a loop. A candidate program is produced, executed, scored, stored, and later reused as context for another attempt. The system can therefore improve even when most individual LLM suggestions are weak.

This distinction also changes what should be analyzed. The central object is no longer the prompt alone, but the full discovery loop: which state is stored, which candidate is sampled, which model action is allowed, which evaluator decides success, and which information is carried forward. AlphaEvolve answers these questions with a fixed evolutionary scaffold. EvoX makes the search strategy adaptive. CORAL gives long-running agents more control over retrieval, local testing, evaluation timing, and memory writing. The report therefore studies agentic discovery as a systems problem in control, memory, and measurement.

1.2 Research Selection

AlphaEvolve was selected because it is the most direct representative of large-scale LLM-guided evolutionary coding for scientific and engineering discovery. It extends the FunSearch style of evaluator-guided program evolution to larger codebases, richer context, state-of-the-art models, multi-metric evaluation, and production systems. It

also provides a strong empirical anchor: a rank-48 algorithm for multiplying two 4×4 complex-valued matrices, improvements to mathematical constructions, and deployed optimizations in Google's compute stack.

EvoX was selected because it identifies a limitation that remains even after AlphaEvolve: the search strategy is usually fixed by humans. EvoX makes the strategy itself an object of evolution. This is a conceptually important step because it moves adaptivity from the candidate programs to the process that generates candidates. It also provides broad evaluation across mathematical, systems, and algorithmic tasks, making it useful for understanding whether adaptive search policies generalize beyond a single benchmark.

CORAL was selected because it pushes the autonomy axis further than the other two papers. It reframes open-ended discovery as autonomous multi-agent evolution rather than fixed evolutionary search. Instead of selecting parents through a hard-coded sampler, it allows agents to inspect shared memory, plan changes, run tests, and externalize knowledge through notes and skills. It is therefore the strongest test case for the claim that discovery systems benefit from long-running agent processes rather than from isolated LLM calls.

1.3 Review Methodology

The report treats the three papers as system papers. That means the analysis emphasizes interfaces, control loops, feedback paths, and failure modes. A reported improvement is interpreted in relation to the evaluator that produced it, the budget under which it was obtained, and the baseline it was compared against. This posture is important because agentic discovery systems can look impressive while still being fragile. A system can overfit to an evaluator, exploit a bug, accumulate noisy memory, or spend so much computation that a simpler baseline would be preferable in deployment.

The review also avoids treating autonomy as an unqualified good. Autonomy can reduce human scaffolding and discover unexpected paths, but it can also reduce predictability. The most mature design is likely to combine autonomous search with strong evaluator isolation, audit trails, typed artifacts, budget controls, and human inspection at decision points where errors are expensive. This balanced view is especially relevant for operating systems and infrastructure contexts, where an optimization that looks good in a benchmark may be unsafe if it cannot be debugged or rolled back.

1.4 Review Contributions

This review makes four contributions. First, it reorganizes the three papers under a common analytical framework centered on autonomy, search adaptivity, memory, evaluation, and deployment cost. Second, it connects the systems to their empirical settings rather than summarizing headline results in isolation. Third, it uses the completed presentation slides as an interpretive guide for the narrative arc from single-agent fixed search to single-agent adaptive search and then to multi-agent adaptive search. Fourth, it identifies open challenges that remain even if the reported results are accepted: evaluator validity, budget fairness, reproducibility, memory governance, and human control.

The review also has a more practical contribution for systems design. It clarifies which pieces of an autonomous discovery system are essential and which are optional. Automated evaluation is essential because it grounds the search. A persistent store of attempts is essential because progress requires reuse. Sophisticated multi-agent organization is not always essential, but it becomes more attractive when the search landscape has many local optima and when different agents can pursue genuinely different strategies.

1.5 Chapter Organization

Chapter 2 introduces a shared technical formulation for evaluator-guided discovery, including how candidate programs, memories, strategies, and agent actions can be modeled. Chapter 3 analyzes AlphaEvolve as a fixed-scaffold evolutionary coding agent. Chapter 4 analyzes EvoX as a system that makes the search strategy itself adaptive. Chapter 5 analyzes CORAL as an autonomous multi-agent framework whose shared memory becomes the coordination mechanism. Chapter 6 synthesizes the three papers and discusses open challenges. Chapter 7 concludes with future directions.

Chapter 2

Background and Preliminaries

2.1 Technical Background

The motivation for autonomous discovery systems comes from a mismatch between the capabilities of current LLMs and the structure of real engineering work. LLMs are strong at local code synthesis, explanation, and pattern completion, but discovery is rarely a single-turn problem. The useful loop is iterative and evidence-driven. A candidate must be executed, measured, compared, and revised. In domains such as compiler optimization, kernel engineering, data-center scheduling, or mathematical construction, the quality of a solution can often be expressed by a scalar or multi-objective evaluator. That makes the domain suitable for agentic search: the model generates candidate programs and the environment provides a grounding signal. The broader intellectual ancestry includes genetic programming, quality-diversity search, learned optimization, and recent LLM-guided evolution [21, 3, 23, 33, 5, 25].

This evaluator-grounded setting is also attractive because it reduces one of the main weaknesses of LLMs. If a model hallucinates a theorem or claims a speedup that does not exist, an external evaluator can reject the candidate. The system does not need the model to be perfectly truthful; it needs the model to produce enough useful variation that selection can amplify the successes. This idea connects LLM agents with classical evolutionary computation, black-box optimization, and program synthesis. It also connects to a larger movement in AI-assisted science, where learned systems have guided mathematical intuition, protein-structure prediction, chemical experimentation, and algorithm discovery [10, 19, 4, 11].

2.1.1 Autonomy, Control, and Memory

The framework used in this report has five dimensions. The first is the decision space: what choices does the system make at each step? AlphaEvolve chooses programs from a database and asks an LLM to propose diffs. EvoX chooses not only candidate programs but also search strategies. CORAL lets agents choose what context to

retrieve, what local tests to run, and what artifacts to write. The second dimension is context representation: whether the system represents history as scored programs, population statistics, or shared memory objects such as notes and skills.

The third dimension is supervision and feedback. All three systems depend on automated evaluation, but they differ in how the feedback is used. AlphaEvolve uses evaluator scores to update a program database. EvoX uses progress over windows to score strategies. CORAL uses scores, logs, local tests, and heartbeat prompts to drive agent reflection and knowledge accumulation. The fourth dimension is generalization: whether the system can handle new tasks, new stages of a search run, or new agents and memory states. The fifth dimension is overhead, including LLM calls, evaluator calls, wall-clock time, memory management, and operational complexity.

Table 2.1: Comparative framework used throughout the report.

Dimension	Guiding question
Decision space	What decisions are made by fixed code, by an LLM, or by an autonomous agent?
Context representation	How are previous attempts, scores, failures, and reusable ideas stored?
Feedback signal	Which evaluator or progress signal selects future behavior?
Generalization	Does the method adapt across tasks, stages, models, or agents?
Overhead	What runtime, cost, memory, and management burden does the method introduce?

2.1.2 Problem Setup

A simplified formalization is useful. Let x denote a task description and let y denote a candidate solution, usually a program. An evaluator E maps (x, y) to a score s and possibly auxiliary feedback f. The goal is to find a candidate $y^\star$ with a high score under a budget of model calls, evaluator calls, or wall-clock time. A discovery system maintains a memory M_t containing previous candidates and feedback. At step t, it constructs a context $\hat{M}_t$, proposes a new candidate y_{t+1}, evaluates it, and updates memory to M_{t+1}.

In a general form, the loop can be written as

$$\hat{M}_t = R(M_t, x), \tag{2.1}$$

$$y_{t+1} \sim G(\cdot \mid x, \hat{M}_t), \tag{2.2}$$

$$(s_{t+1}, f_{t+1}) = E(x, y_{t+1}), \tag{2.3}$$

$$M_{t+1} = U(M_t, y_{t+1}, s_{t+1}, f_{t+1}), \tag{2.4}$$

where R is a retrieval or context-construction rule, G is the generator or agent policy, E is the external evaluator, and U is the memory update rule. The three papers differ mainly in which of these functions are fixed and which are delegated to adaptive models or agents.

The common mathematical objective is budgeted evaluator-guided search:

$$y_T^\star \in \arg \max_{y \in \{y_1, \ldots, y_T\}} S(E(x, y)), \tag{2.5}$$

where S extracts or aggregates the evaluator score and T denotes the official evaluation budget. For minimization tasks, the sign of S is reversed or the comparison operator is changed. This compact objective hides the main engineering choices: candidate representation, context retrieval, local testing, memory update, and whether the search policy itself can change.

Fixed evolutionary search specifies most of these operations externally. A sampler chooses parents, the LLM proposes a mutation, the evaluator scores it, and a database update rule stores the result. Meta-evolution expands the formalization by adding a strategy variable S_t that governs how contexts are built. Autonomous multi-agent evolution expands it further by introducing several agents $a_1, \ldots, a_n$, each with private workspace state and access to shared memory. The central design problem becomes deciding which parts of retrieve, propose, evaluate, and update should be fixed, learned, or delegated to agents.

2.1.3 Method Families

The three focal papers instantiate the common loop at different levels of autonomy. AlphaEvolve keeps R and U largely system-defined through prompt sampling and a program database; the LLM mainly implements a mutation operator inside G. EvoX introduces a strategy variable S_t so that the context-construction mechanism changes over time. CORAL shifts more of R, G, and U into autonomous agents, while keeping the evaluator and execution safeguards external.

The same distinction can be stated in state-variable terms. AlphaEvolve's state is a program database containing candidates, scores, and feedback, and its update step inserts new evaluated programs for future prompting. EvoX extends the state with a strategy history, so that a candidate generator is conditioned not only on prior solutions but also on past strategy performance. CORAL replaces a single database abstraction with shared memory containing attempts, notes, and skills; the agents themselves decide which memory items to retrieve, how to test a candidate locally, and what evidence to write back after evaluation. This prose distinction is used throughout the report to avoid treating all three systems as minor variants of the same evolutionary loop.

Table 2.2: Instantiation of the common discovery loop in the three reviewed systems.

System	Retrieval/context	Proposal policy	Memory update
AlphaEvolve	Prompt sampler selects prior programs and feedback	LLM ensemble proposes structured code diffs	Program database stores scored candidates
EvoX	Search strategy S_t selects parents, operators, and inspiration	LLM generates candidates under the active strategy	Solution database and strategy history are both updated
CORAL	Agents inspect shared attempts, notes, and skills	Agents plan, edit, test, and submit candidates	Agents write attempts, notes, and skills to shared memory

2.2 Related Work

The three papers build on several lines of work. FunSearch introduced the idea of using LLMs inside evolutionary loops for mathematical discovery, where generated programs are selected by external evaluation [39]. AlphaEvolve scales that idea to larger codebases and practical infrastructure [34]. This line also relates to AlphaTensor, symbolic optimizer discovery, and earlier LLM-guided code evolution [11, 7, 15]. Other systems such as OpenEvolve, ShinkaEvolve, GEPA, PromptBreeder, and EvoPrompt explore different selection rules, diversity mechanisms, prompt evolution, and feedback summaries [1, 24, 40, 12, 14].

The papers also connect to autonomous agents and multi-agent collaboration. Systems such as the AI Scientist and AI Co-Scientist decompose research workflows into agentic steps, while multi-agent software frameworks use role assignment and structured communication [13, 16, 29, 26, 6, 37, 46]. Agent work on reasoning-action loops, verbal reinforcement, iterative self-feedback, memory, and software-engineering interfaces provides additional context for CORAL's long-running agent design [49, 41, 30, 36, 48, 44]. CORAL differs by emphasizing horizontal parallelism and shared memory rather than predefined vertical roles. Finally, the systems connect to learned optimization and meta-learning because EvoX treats the search process itself as an object that can be optimized [2, 32, 27].

2.3 Evaluation Methodology

The evaluation settings in the three papers are diverse. AlphaEvolve evaluates on fast matrix multiplication, constructive mathematics, and engineering tasks in Google's compute stack. EvoX evaluates on mathematical optimization, system optimization, ALE-Bench-Lite, and Frontier-CS. CORAL evaluates on mathematical and systems

tasks, Polyominoes, and Anthropic's kernel engineering task. These benchmarks share an important property: they provide automated scoring. Without such scoring, the evolutionary loop would require human judgment and would be much harder to scale.

At the same time, automated evaluation is a source of risk. The evaluator must measure the property that actually matters. If it is incomplete, an agent may optimize the wrong thing. If it contains a bug, the agent may exploit the bug. If the benchmark workload is narrow, the discovered program may not generalize. The CORAL paper explicitly reports evaluator corrections for several systems tasks, which is a useful reminder that evaluation infrastructure is part of the scientific claim, not a neutral background detail.

2.3.1 Benchmark Families

AlphaEvolve's matrix multiplication results are tied to a long tradition of tensor-rank formulations. The reported rank-48 algorithm for 4×4 complex-valued matrix multiplication is significant because it improves over the recursive use of Strassen's algorithm in that setting. Its mathematical construction tasks measure whether the system can discover objects with better certified properties, such as packings or bounds in combinatorics. Its engineering tasks measure whether small code changes can produce meaningful improvements in repeatedly executed infrastructure components.

EvoX's benchmarks are designed to test whether strategy adaptation helps across heterogeneous search landscapes. Signal processing illustrates phase changes in the search: random sampling and greedy refinement work early, but later improvement requires stratified multi-objective sampling and structural variation. Systems tasks such as GPU placement, cloud transfer, LLM-SQL, and transaction scheduling test whether the same adaptive mechanism works outside geometric or mathematical construction. Frontier-CS, ALE-Bench-Lite, and SkyDiscover add broader algorithmic and systems diversity [31, 18, 28].

CORAL's benchmarks test a different hypothesis. The key claim is not only that agents can find high-scoring solutions, but that autonomy and shared memory improve the trajectory. Kernel engineering is a particularly useful stress test because the objective is clear, the optimization landscape is difficult, and progress depends on detailed low-level insight. The multi-agent results are therefore evidence for knowledge diffusion and exploration diversity, not merely for a stronger mutation operator. This is closely aligned with recent interest in agentic systems research and kernel-generation benchmarks [8, 35].

2.3.2 Metrics and Budgets

The papers use several kinds of metrics. Some are direct objective scores, such as tensor rank, speedup, cycle count, packing score, or scheduling score. Others are trajectory metrics, such as improvement rate, number of evaluations to reach a record, or wall-clock time to convergence. EvoX also uses strategy progress over a window to decide whether to evolve the search strategy. CORAL reports memory-related trajectory statistics such as knowledge creation, knowledge access, and read-to-improvement rate.

These metrics should not be collapsed into a single notion of "better." A method with a higher final score may be more expensive. A method with a lower number of evaluations may spend more time reasoning between evaluations. A method with better wall-clock time may depend on more parallel agents. A mature comparison must therefore ask what resource is being optimized: final frontier, sample efficiency, wall-clock efficiency, monetary cost, human interpretability, or operational simplicity.

2.4 Review Boundaries

The report focuses on three papers and does not attempt to survey every LLM-based optimization system. It also relies on the papers' reported results rather than rerunning experiments. This limitation matters because many claims depend on proprietary models, private infrastructure, or task-specific evaluator details. The review therefore treats reported numbers as evidence within their experimental context, not as universal rankings.

The report also focuses on evaluator-guided discovery tasks. It does not cover scientific domains where the evaluator requires wet-lab experiments, human peer review, or long simulation times. The methods may inspire such domains, but their current strongest evidence comes from tasks where evaluation is automated and relatively fast.

A further boundary is the level of system detail considered. The report discusses prompts, program databases, strategy loops, shared memories, and evaluation protocols, but it does not attempt to reconstruct all implementation choices that are omitted from the papers. This matters because scheduling policy, evaluator caching, invalid-program filtering, and model-sampling parameters can materially affect the observed search trajectory.

The comparison is therefore architectural rather than fully experimental. AlphaEvolve, EvoX, and CORAL are treated as three design points for autonomous discovery systems: fixed evolutionary scaffolding, adaptive search-policy evolution, and agent-controlled multi-agent search. This framing supports cross-paper analysis without implying that the reported numbers are directly interchangeable across tasks or compute budgets.

Autonomy is also separated into decision types rather than treated as a single scalar. A system may be autonomous in proposing code but not in defining objectives; autonomous in adapting search strategy but not in changing the evaluator; or autonomous in local tool use while still constrained by a manager. Keeping these distinctions separate prevents the comparison from rewarding surface-level agency over the more important question of which decisions are actually delegated to the system.

2.4.1 Threats to Comparison

The first threat is benchmark mismatch. AlphaEvolve, EvoX, and CORAL do not evaluate on identical task sets under identical budgets. The second threat is model mismatch. Different underlying LLMs, including Gemini, GPT-5, Claude Opus, and others, can change the strength of the generator. The third threat is budget mismatch. A multi-agent system may use more parallel compute or more wall-clock management than a single-agent baseline. The fourth threat is evaluator mismatch: a result is only as reliable as the grader that produced it.

The review handles these threats by comparing design principles and reported trajectories, not only final scores. The strongest conclusions are qualitative: fixed search is powerful but rigid, strategy adaptation can break stagnation, and shared memory can help agents transfer discoveries. The exact magnitude of improvement should be interpreted more cautiously.

2.5 Summary

This chapter established the common vocabulary for the report. Autonomous discovery systems search over candidate programs under evaluator feedback. Their performance depends on the design of the decision space, memory, feedback use, generalization target, and overhead. The next three chapters apply this framework to AlphaEvolve, EvoX, and CORAL.

Chapter 3

AlphaEvolve: Evolutionary Coding Agent for Discovery

3.1 Background and Motivation

AlphaEvolve addresses a direct and ambitious problem: can an LLM-based system discover better algorithms and engineering code when candidate solutions can be automatically evaluated? The paper argues that discovery requires more than fluent code generation. Models must be embedded in a process that tests proposals, stores what worked, and uses feedback to guide later attempts. AlphaEvolve therefore treats the LLM as one component in an evolutionary system rather than as the whole system.

The motivation is reinforced by the range of tasks. The system is applied to matrix multiplication, constructive mathematics, data-center scheduling, kernel optimization, hardware arithmetic circuits, and attention kernels. These are not toy examples in the usual sense. They are domains where small improvements can matter because the code is executed repeatedly or because the mathematical result improves a known frontier.

AlphaEvolve is therefore broader than a FunSearch-style search over one short function. The editable object can be a larger code file, the evaluator can expose multiple metrics and execution feedback, and the final artifact may be a production heuristic or kernel rather than a compact mathematical expression. This distinction matters for engineering workloads: the loop is the same, but the acceptance path changes from mathematical verification to testing, review, rollout, and rollback. It also places AlphaEvolve in a lineage that includes classical matrix-multiplication algorithms, reinforcement-learning algorithm discovery, and systems optimization under production constraints [42, 17, 22, 11, 43].

Table 3.1: Capability shift from FunSearch-style search to AlphaEvolve.

Dimension	FunSearch-style setting	AlphaEvolve setting
Editable object	A single short function	A complete code file with larger surrounding context
Programming scope	Python-centric mathematical snippets	General code tasks, including production engineering settings
Evaluation signal	Usually a single scalar objective	Multiple metrics and richer execution feedback
Human role	Define task and evaluator	Define task, evaluator, seed code, validation path, and deployment gate
Best fit	Compact mathematical constructions	Algorithms, kernels, scheduling heuristics, and system optimizations

3.2 AlphaEvolve Method

AlphaEvolve is not a single prompt template, nor is it simply an LLM that writes code and then receives a score. The paper presents it as an autonomous coding pipeline organized around an evolutionary controller. A human scientist or engineer specifies the task, provides an initial program, marks the parts of the code that may be changed, supplies evaluation code, and optionally adds prompt templates, model choices, and background material. After that setup, AlphaEvolve runs a closed loop: it samples parent programs and inspirations from an evolutionary database, constructs a rich prompt, asks an LLM ensemble to propose code changes, applies those changes to form a child program, evaluates the child, and registers the new program and feedback in the database. The division of labor is therefore deliberate: the human defines what should count as progress, while the system searches over how to obtain that progress.

The method is best understood as a whole system with six interacting layers. The task interface defines the search space and the objective. The prompt sampler decides which experience from previous trials is shown to the model. The LLM ensemble acts as a learned mutation operator over code. The evaluator pool turns generated programs into scores, outputs, and diagnostic feedback. The program database maintains evolutionary memory and controls which ideas are resurfaced. Finally, the distributed controller keeps these components running asynchronously so that expensive evaluations and model calls can proceed in parallel. This organization explains why AlphaEvolve can operate on larger and more varied problems than earlier FunSearch-style systems: the LLM supplies creative edits, but the system-level scaffold supplies memory, selection, validation, and throughput.

Table 3.2: AlphaEvolve components and their design roles.

Component	Role	Design implication
Task interface	Defines evolve blocks, evaluation code, and optional context	Determines the candidate representation and what counts as improvement
Prompt sampler	Builds context from parents, inspirations, task material, and evaluation results	Controls what evidence and search trajectory the LLM sees
LLM ensemble	Produces structured code edits or full replacements	Serves as a learned mutation operator with both breadth and quality
Evaluator pool	Executes generated programs and returns one or more metrics	Grounds the search in automatic tests rather than model preference alone
Program database	Stores evaluated programs, scores, outputs, and feedback	Balances exploitation of strong programs with diversity-preserving exploration
Distributed controller	Coordinates sampling, generation, evaluation, and insertion asynchronously	Maximizes discovery throughput under a fixed compute budget

3.2.1 Method Formulation

Let $\mathcal{Y}$ denote the space of candidate programs reachable from the initial code by edits to the marked evolve blocks or, in short-code settings, by complete replacement of the evolved code. AlphaEvolve requires a user-provided evaluator

$$h : \mathcal{Y} \to \mathbb{R}^m, \tag{3.1}$$

where $h(y)$ is a dictionary or vector of scalar metrics to be maximized. This evaluator is usually exposed through a fixed-signature `evaluate` function. For a simple mathematical construction, it may check constraints and return a size or bound. For a systems task, it may run benchmarks, simulations, randomized searches, or even training jobs. The key point is that the generated artifact is judged by executable evidence rather than by the LLM's own self-assessment.

At iteration t, the database D_t contains evaluated programs and their feedback:

$$D_t = \{(y_i, h(y_i), o_i, f_i)\}_{i=1}^{t}. \tag{3.2}$$

Here o_i denotes program outputs or rendered evaluation results, and f_i denotes optional diagnostic feedback or metadata. The controller samples a parent and optional

inspirations,

$$(y_{\text{par}}, I_t) \sim \text{Sample}(D_t), \tag{3.3}$$

builds a prompt $p_t = \text{Prompt}(y_{\text{par}}, I_t, x)$, asks the LLM ensemble for a structured diff $\delta_t \sim G(\cdot \mid p_t)$, applies the diff to obtain $y_{t+1} = \text{Apply}(y_{\text{par}}, \delta_t)$, evaluates $h(y_{t+1})$, and inserts the result into D_{t+1}. In compact form,

$$D_{t+1} = D_t \cup \{(y_{t+1}, h(y_{t+1}), o_{t+1}, f_{t+1})\}. \tag{3.4}$$

This formulation makes clear why AlphaEvolve is more than prompt engineering. The LLM is the proposal distribution, but selection is performed by executable evaluation, and memory is maintained by the evolutionary database. The system can optimize one score or multiple scores. Even when one score is the ultimate target, auxiliary metrics can be useful because they keep structurally different programs visible to the sampler and can expose routes to later improvements.

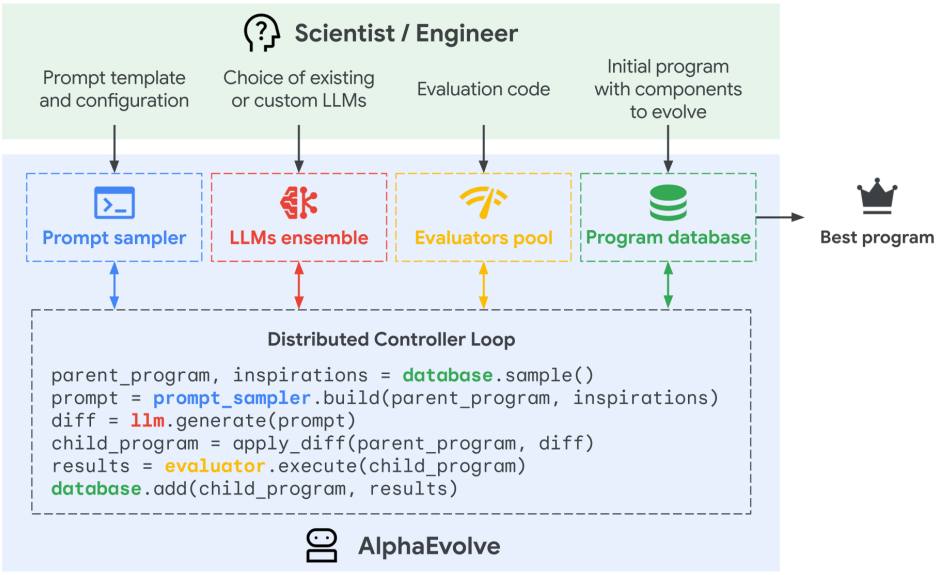


Figure 3.1: AlphaEvolve system architecture with four key components [34].

3.2.2 Prompt Sampler

The prompt sampler turns evolutionary memory into model context. It selects a parent program, optional inspiration programs, their scores, rendered evaluator outputs, and user-provided context such as task instructions, equations, code snippets, or literature. Its technical role is to control the evidence shown to the LLM: exploitation comes from showing strong parents, exploration comes from mixing diverse inspirations, and meta-prompt evolution lets the system co-evolve parts of the instruction context itself.

3.2.3 Creative Generation

The LLM ensemble is AlphaEvolve's learned mutation operator. The reported system combines lower-latency and stronger Gemini models so that the loop gets both throughput and occasional high-quality proposals. Unlike systems that evolve a single short function, AlphaEvolve can propose coordinated edits across larger files, including algorithmic logic, losses, optimizers, sweeps, and systems heuristics. Invalid or weak proposals are filtered by execution; useful proposals survive through database selection.

3.2.4 I/O Interface

```
<<<<<<< SEARCH
   self._block1 = ResNetBlock(num_channels)
   self._block2 = ResNetBlock(num_channels * 2, stride=2)
   self._block3 = ResNetBlock(num_channels * 4, stride=2)
=======
   self._block1 = ResNetBlock(num_channels)
   self._block2 = ResNetBlock(num_channels, stride=1)
   self._block3 = ResNetBlock(num_channels * 2, stride=2)
   self._block4 = ResNetBlock(num_channels * 2, stride=1)
   self._block5 = ResNetBlock(num_channels * 4, stride=2)
   self._block6 = ResNetBlock(num_channels * 4, stride=1)
>>>>>>> REPLACE
```

Figure 3.2: AlphaEvolve-style structured LLM output for automatic search/replace editing before evaluation.

The input interface uses explicit evolve blocks, such as `EVOLVE-BLOCK-START` and `EVOLVE-BLOCK-END`, to define what the model may edit while leaving the surrounding program skeleton stable. The starting program must run, but it can be simple. This boundary lets users choose the right abstraction: a raw construction, a constructor function, a search algorithm, or a co-evolution of partial solutions and search procedures. The output interface is equally strict: the LLM emits search/replace diffs or, for short programs, full replacements, so the controller can apply edits and evaluate child programs without manual interpretation.

3.2.5 Program Database

The program database is both archive and selection policy. Each entry stores a program, scores, outputs, and feedback. Sampling only the current best program would collapse diversity, while random replay would waste evaluation budget. AlphaEvolve therefore uses an evolutionary database inspired by MAP-Elites and island models: strong candidates are resurfaced, but distinct niches and lineages are preserved so that later prompts can recombine useful ideas rather than converge immediately to one family.

MAP-Elites is a quality-diversity algorithm. Instead of maintaining one global champion, it partitions the search space by behavioral or feature dimensions and

stores the best known candidate, or elite, in each cell. A parent is sampled from this archive, mutated to produce an offspring, evaluated for both fitness and feature coordinates, and inserted only if it improves the elite in its target cell. This matters for AlphaEvolve because programs with different score profiles or behaviors may contain reusable ideas even when they are not the current best overall solution.

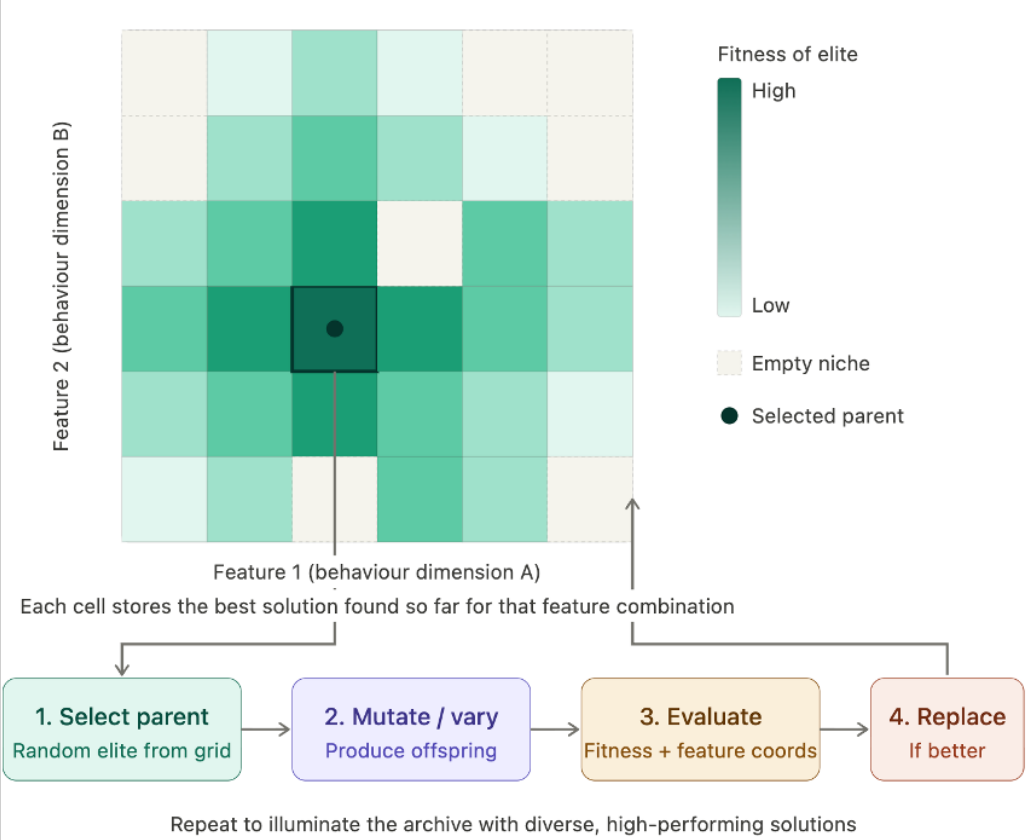


Figure 3.3: MAP-Elites archive and replacement loop for diverse high-performing candidates. Figure created by the report author.

3.2.6 Execution Feedback

Execution feedback is the supervision signal. The evaluator returns scalar metrics and diagnostics, while cascaded evaluation filters candidates on cheap tests before expensive ones. Parallel evaluation lets searches, simulations, or training jobs run across a cluster, and LLM-graded auxiliary criteria can be added for properties such as simplicity. The controller runs generation, evaluation, and database insertion asynchronously, optimizing throughput rather than the latency of one candidate. The credibility of the whole method depends on this feedback being valid: exact verification and production validation are what separate discoveries from benchmark artifacts.

3.3 Evaluation and Results

3.3.1 Matrix Multiplication

The headline mathematical benchmark is fast matrix multiplication. AlphaEvolve searches for algorithms corresponding to low-rank tensor decompositions. The most notable result is an algorithm for multiplying two 4×4 complex-valued matrices using 48 scalar multiplications, improving over the 49 multiplications obtained by recursively applying Strassen's method in this setting. The paper also reports improvements for 14 matrix multiplication targets and verifies that discovered algorithms are exact. The result is meaningful because matrix multiplication has a long history in which single-rank improvements can affect both theoretical complexity and practical kernel design [42, 22, 20].

AlphaEvolve is also evaluated on more than 50 mathematical problems involving constructions. The system reportedly matches the best known constructions on roughly three quarters of them and improves the state of the art on roughly one fifth. These include problems in overlap bounds, packing, Heilbronn-type configurations, and kissing numbers. The important feature is that candidates can be checked by code, even when the mathematical object itself is not a program in the final presentation.

Table 3.3: Selected AlphaEvolve result categories.

Category	Reported result	Interpretation
Matrix multiplication	Rank-48 algorithm for 4×4 complex-valued matrices	Frontier improvement in a classic algorithmic problem
Constructive mathematics	SOTA improvements on about one fifth of tested problems	Evidence that code search can discover mathematical objects
Data-center scheduling	Recovers about 0.7 percent stranded compute fleet-wide	Small interpretable heuristic has large operational value
Kernel and circuit optimization	Speedups in Gemini kernels, TPU circuits, and attention	Shows applicability to infrastructure layers

3.3.2 Mathematical Constructions

The matrix multiplication results show that AlphaEvolve is not merely tuning parameters. The discovered programs can introduce non-trivial changes to optimization procedures, loss functions, initialization, and hyperparameter schedules. Because the final tensor decompositions can be verified exactly, the result is stronger than a

benchmark-only speedup. It is a new certified algorithmic construction for specific tensor shapes.

Table 3.4: Matrix-multiplication tensor-rank results reported by AlphaEvolve.

Tensor target $\langle m, n, p \rangle$	Previous best	AlphaEvolve	Change
$\langle 2, 4, 5 \rangle$	33	32	-1
$\langle 2, 4, 7 \rangle$	46	45	-1
$\langle 3, 4, 6 \rangle$	56	54	-2
$\langle 3, 4, 7 \rangle$	66	63	-3
$\langle 4, 4, 4 \rangle$	49	48	-1
$\langle 4, 4, 8 \rangle$	98	96	-2
$\langle 5, 5, 5 \rangle$	93	93	0

The systems results are significant for a different reason. A data-center scheduling heuristic or a kernel optimization does not need to be mathematically surprising to be valuable; it needs to be correct, interpretable, and repeatedly useful. The reported Borg scheduling heuristic is short enough to inspect, which matters in production. The Gemini kernel optimization demonstrates a recursive quality of the field: an LLM-based system can optimize parts of the infrastructure used to train LLMs.

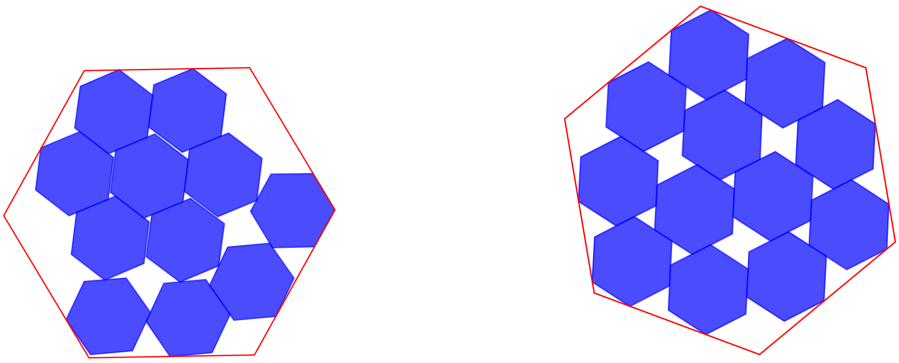


Figure 3.4: AlphaEvolve constructions for packing 11 and 12 unit hexagons inside a regular hexagon, with side lengths 3.931 and 3.942 [34].

3.3.3 Engineering Results

The engineering results show why a discovery agent can matter even when the discovered artifact is small. AlphaEvolve found a short scheduling heuristic for Google's Borg cluster-management setting that the paper reports as continuously recovering on average 0.7 percent of fleet-wide stranded compute. It also improved matrix-multiplication kernel heuristics used in Gemini training, with a reported 23 percent average kernel speedup across the optimized kernels and about a 1 percent reduction in overall training time. For attention, the paper reports a 32 percent speedup on

a FlashAttention kernel through optimization of XLA-generated intermediate representation [43, 9].

Table 3.5: Selected AlphaEvolve engineering results reported in the paper.

Application	Reported result	Why it matters
Google Borg scheduling	0.7% average recovery of fleet-wide stranded compute	Small interpretable heuristics can have large infrastructure impact.
Gemini kernel optimization	23% average kernel speedup and about 1% overall training-time reduction	Repeated low-level improvements compound in large training pipelines.
FlashAttention kernel	32% speedup	The approach can operate below ordinary Python-level code, on compiler IR.
TPU arithmetic circuit	Functionally equivalent simplification validated by hardware experts	Evolution can be useful in hardware-adjacent engineering when correctness is checkable.

```python
def alpha_evolve_score(required, free):
  cpu_residual = required.cpu / free.cpu
  mem_residual = required.mem / free.mem

  return -1.0 * (cpu_residual + mem_residual +
                 mem_residual / cpu_residual +
                 cpu_residual / mem_residual)
```

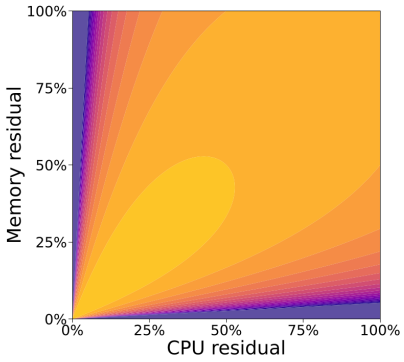


Figure 3.5: Borg scheduling heuristic discovered by AlphaEvolve and a visualization of its scoring surface; yellow regions are high scores and purple regions are low scores [34].

These results also mark the boundary of AlphaEvolve's autonomy. Once the task is formalized, the system can search effectively, but the human still decides the editable code regions, the evaluator, and the acceptability criteria for deployment. EvoX and CORAL can therefore be read as relaxing different parts of this scaffold rather than replacing the basic evaluator-grounded insight.

3.4 Discussion

3.4.1 Cost and Deployment

The deployment interpretation of AlphaEvolve is strongest in tasks where evaluator costs are acceptable and improvements compound. A 0.7 percent recovery of stranded compute may sound small, but at fleet scale it is operationally meaningful. A 1 percent reduction in training time for a large model can represent substantial savings. The cost of the search must therefore be compared with the lifetime value of the discovered improvement.

The overhead comes from LLM sampling, candidate execution, and database management. AlphaEvolve mitigates this overhead through asynchronous execution and evaluation cascades. Cheap tests can filter bad candidates before expensive evaluation. This pattern is important for future systems: autonomy is most practical when evaluation is staged and when the system can fail fast.

A deployment lesson is that the evaluator is not only a scoring function but also a control surface. By deciding which failures are cheap, which constraints are hard, and which metrics are recorded for later prompts, the designer shapes the search space seen by the LLM. AlphaEvolve's success therefore depends on a tight coupling between code generation, execution feedback, and database selection rather than on generation quality alone [34].

3.4.2 Strengths

AlphaEvolve's first strength is grounding. Every candidate is scored by an external evaluator, so the system is less dependent on the LLM's self-assessment. Its second strength is expressivity. By evolving code, it can search over algorithms, constructors, heuristics, and optimization procedures. Its third strength is scalability. The use of an LLM ensemble, asynchronous evaluation, and a program database makes the system suitable for large search campaigns. Its fourth strength is interpretability in some deployed cases: short discovered heuristics can be inspected and maintained by engineers.

The system's fifth strength is its demonstration breadth. A single paper reports progress across mathematics and infrastructure. That breadth supports the claim that LLM-guided evolution is a general pattern rather than a narrow trick for one benchmark.

3.4.3 Limitations

The main limitation is dependence on automated evaluation. If the score is slow, noisy, or misaligned with the real objective, the system becomes less reliable. A second limitation is that the search scaffold is mostly fixed. MAP-Elites-style sam-

pling and island models can support diversity, but they do not by themselves learn when the search landscape has changed. A third limitation is reproducibility. The strongest results may depend on frontier models, large compute budgets, and internal engineering infrastructure.

There is also a human-interface limitation. The user still has to decide what code can be edited, how to score it, and what constraints must be enforced. These decisions are often the hardest part of applying the system safely. AlphaEvolve reduces the cost of search after the task has been formalized; it does not remove the need for careful formalization.

The limitation is less severe when the editable surface is narrow and the acceptance tests are strong. In those settings, the system can explore aggressively while humans retain control over the surrounding software boundary. It becomes more problematic when the desired improvement depends on ambiguous product goals, long-term maintainability, or implicit domain assumptions that are difficult to encode in an evaluator.

3.4.4 Position in the Review

In the three-paper arc, AlphaEvolve is the foundation. It shows that LLMs can be useful mutation engines when embedded in an evaluator-guided loop. It also establishes the importance of program databases, structured prompts, execution feedback, and multi-metric scoring. Without this foundation, later claims about adaptive strategy or multi-agent memory would be less convincing.

At the same time, AlphaEvolve leaves open the question of how much of the loop should be fixed. Its success motivates the next step: if programs can be evolved, perhaps the strategy for evolving programs can also be evolved. That is precisely the move made by EvoX.

3.5 Summary

AlphaEvolve demonstrates that evaluator-grounded LLM code evolution can produce scientific and practical discoveries. Its core strength is the combination of expressive code generation with hard feedback from execution. Its main limitation is that the surrounding search strategy remains largely human-designed. The next chapter analyzes EvoX, which treats that strategy as an adaptive object.

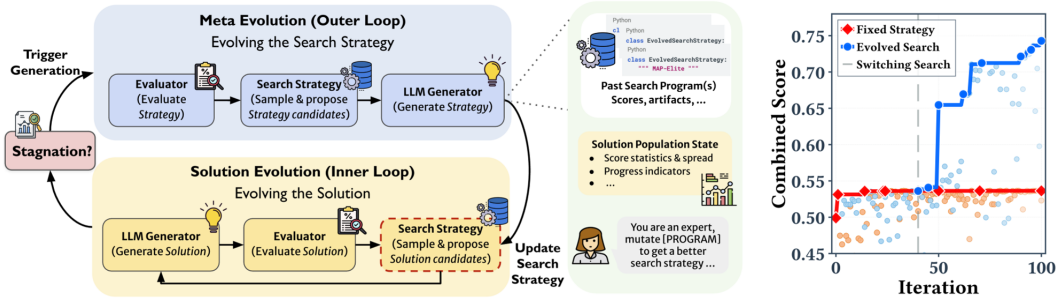
Chapter 4

EvoX: Meta-Evolution for Automated Discovery

4.1 Background and Motivation

EvoX starts from a simple observation: a fixed evolutionary strategy is unlikely to be optimal throughout a search run. Early in a run, broad exploration may be useful. Later, the population may contain strong candidates that need careful refinement. In some tasks, progress requires a structural change rather than local tuning. If the system uses the same sampling rule and variation balance from beginning to end, it may stagnate after the initial easy gains. This observation echoes work on learned optimizers and hyper-heuristics: the policy that chooses updates can itself be a learned or evolved object [2, 5, 50].

This critique applies directly to systems like AlphaEvolve, OpenEvolve, and related frameworks. They maintain candidate populations and use LLMs to generate variants, but their search behavior is largely governed by fixed knobs. EvoX reframes the problem as meta-evolution: the system evolves both solutions and the search strategy that generates solutions.



(a) Overall system architecture

(b) Evolved search (blue) breakthroughs

Figure 4.1: EvoX two-level loop: an inner loop evolves solutions while an outer loop evolves the search strategy when progress stagnates [27].

4.2 EvoX Method

4.2.1 Problem Formulation

EvoX formalizes a search strategy as the mechanism that constructs the next LLM input from the current solution database. The strategy chooses parent candidates, optional inspiration examples, and a variation operator. The variation operator may request local refinement, structural variation, or free-form modification. This abstraction is useful because it isolates the decision layer that many earlier systems hard-code.

The paper writes candidate evaluation as

$$E(x) \to (s(x), a(x)), \tag{4.1}$$

where $x \in \mathcal{X}$ is a candidate solution, $s(x) \in \mathbb{R}$ is the score, and $a(x)$ denotes auxiliary artifacts such as logs or traces. After t evaluations, the solution database is

$$D_t = \{(x_i, s_i, a_i)\}_{i=1}^{t}, \qquad D_0 = \emptyset. \tag{4.2}$$

A search strategy $S \in \mathcal{S}$ constructs the generation context

$$C_S(D_t) \to (x_{\text{par}}, \pi, I), \tag{4.3}$$

where x_{par} is a parent candidate, π is the requested variation operator, and $I \subseteq D_t$ is an optional inspiration set. The solution generator then samples

$$x' \sim G_{\text{sol}}(\cdot \mid x_{\text{par}}, \pi, I). \tag{4.4}$$

The optimization objective under a budget T is to maximize the best score in the final database:

$$\max_{(x,s,a) \in D_T} s. \tag{4.5}$$

The solution generator remains an LLM. What changes is the control policy around it. Instead of always sampling from the population in the same way, EvoX can generate a new strategy conditioned on previous strategies, their observed performance, and the current population state. The strategy is itself represented as code implementing the required sampling interface.

4.2.2 Solution Evolution

The inner loop evolves candidate solutions under the current strategy for a window of iterations. During that window, the system records progress, such as the improvement in best score. The outer loop then evaluates whether the strategy is still

productive. If progress falls below a stagnation threshold, the strategy generator proposes a replacement strategy. Valid strategies are stored along with state descriptors and scores, forming a strategy database.

Table 4.1: EvoX two-level loop.

Phase	Function
Solution evolution	Use the active strategy to select parents, choose variation, generate candidates, and evaluate them.
Progress monitoring	Measure improvement over the search window and record strategy performance.
Strategy evolution	On stagnation, ask the LLM to rewrite the strategy using population state and prior strategy history.

This structure makes EvoX different from simply changing a few hyperparameters. The strategy can encode conditional behavior, usage penalties, score-band sampling, multi-objective sampling, or operator biases. Because the strategy is code, it can express task-specific logic while preserving the required interface.

4.2.3 Strategy Evolution

EvoX deploys a strategy for a window of W solution-evolution iterations. Let s_{start} and s_{end} be the best scores before and after the window, and define

$$\Delta = s_{\text{end}} - s_{\text{start}}. \tag{4.6}$$

The paper scores the deployed strategy using

$$J_t = \frac{\Delta \log(1 + s_{\text{start}})}{\sqrt{W}}, \tag{4.7}$$

then stores a tuple $(S_t, \phi(D_t), J_t)$ in the strategy history, where $\phi(D_t)$ is a population-state descriptor. On stagnation, the strategy generator proposes

$$S' \sim G_{\text{str}}(\cdot \mid H, \phi(D_t)), \tag{4.8}$$

where H is the history of strategies, states, and strategy scores. The key technical move is that the search policy is conditioned on both past strategy performance and the current population state, rather than being a static hand-written sampler.

4.3 Evaluation

4.3.1 Tasks and Baselines

EvoX evaluates across nearly 200 tasks. The mathematical optimization tasks include circle packing, Heilbronn variants, min-max-min distance, autocorrelation inequalities, and signal processing. The systems tasks include expert-parallel load balancing, GPU placement, LLM-SQL, cloud transfer, transaction scheduling, and telemetry repair. The algorithmic benchmarks include ALE-Bench-Lite and Frontier-CS [18, 31].

This breadth is important because the paper's main claim is about adaptive search behavior. A single task could be solved by a lucky hand-coded strategy. A broad task suite tests whether the meta-evolution mechanism can repeatedly discover useful search policies. The paper reports comparisons against OpenEvolve, GEPA, ShinkaEvolve, and in some cases AlphaEvolve or human best results.

4.3.2 Comparison Protocol

The baselines represent different fixed or partially adaptive approaches. OpenEvolve is an open implementation of AlphaEvolve-like evolutionary search. GEPA emphasizes reflective prompt or program improvement along Pareto-style considerations. ShinkaEvolve includes adaptive sampling elements but still keeps key strategy knobs fixed. AlphaEvolve provides a closed-source reference on comparable mathematical tasks. EvoX is evaluated under fixed iteration budgets for open frameworks, which helps isolate the value of strategy evolution. The comparison also sits beside newer long-horizon evolution systems that try to stabilize progress over many search phases [47, 45].

The evaluation scenarios also distinguish between mean and best scores. This distinction is important because evolutionary search can have high variance. A best score measures the frontier reached by at least one run; a mean score indicates reliability. EvoX's claims are stronger when both improve.

4.3.3 Main Results

The signal-processing case study illustrates the mechanism most clearly. A fixed MAP-Elites-style strategy improves early and then stagnates. EvoX begins with random and greedy behavior, then shifts to stratified multi-objective sampling, then to UCB-guided structural variation, and finally to local refinement. The reported final score is 34.1 percent higher than the static baseline. The important lesson is not only the final number; it is the phase structure of the trajectory.

On mathematical optimization, EvoX reports best or tied-best results on most tasks and often matches or exceeds AlphaEvolve on comparable problems within the evaluation budget. On systems tasks, it obtains strong mean or best scores

under GPT-5 and Gemini-3.0-Pro. On ALE-Bench-Lite and Frontier-CS, it improves average or median performance over open baselines. These results support the central claim that adaptive strategy selection can matter across domains.

Table 4.2: Selected EvoX headline results reported in Table 1 of the paper.

Task	Human best	Prior AI / AlphaEvolve	EvoX
Circle-Packing (↑)	2.634	2.635	2.636
MinMaxMinDist (↓)	4.16584	4.16579	4.16578
Cloud Transfer (↓)	626.24	645.72	623.69
GPU Placement (↑)	21.89	26.26	30.52
Transaction Scheduling (↑)	2724.8	4329	4347.8
Frontier-CS median (↑)	–	56.2	75.5

Table 4.3: Additional EvoX system-optimization results reported in the paper.

Task	Human best	AI best / baseline	EvoX	Direction
EPLB	0.1265	0.1445	0.1453	↑
PRISM	21.89	26.26	30.52	↑
LLM-SQL	0.6920	0.7155	0.7298	↑
Cloudcast	626.24	645.72	623.69	↓
Transaction Scheduling	2724.8	4329.0	4347.83	↑
Telemetry Repair	0.8222	0.9515	0.9520	↑

4.3.4 Strategy Trajectories

EvoX's most interesting generalization target is not a new model or a new dataset, but a new state within the same search process. The system asks: given this population, this score distribution, and this recent progress pattern, what strategy should be used next? This is a form of online adaptation. The strategy that works at iteration 10 may not work at iteration 70.

This point makes EvoX especially relevant for open-ended discovery. In many optimization landscapes, the first improvement is easy because the seed is weak. Later improvements require identifying bottlenecks, combining partial ideas, or changing the representation. Strategy evolution gives the system a way to respond to these phase changes without a human manually retuning the search.

Table 4.4: EvoX signal-processing strategy trajectory reported in the paper.

Search phase	Strategy type	Window	Score gain	Interpretation
Initial state	Baseline population	–	score 0.499	Starting point before adaptive strategy improvements.
Exploration	Random sampling	11	+0.03	Broad variation is useful before the system knows which structures matter.
Early exploitation	Greedy search	10	+0.01	The system begins to reuse high-scoring candidates.
Mid-run restructuring	Stratified multi-objective structural search	19	+0.12	Strategy evolution finds a more productive source of structural variation.
Focused variation	UCB structural search	16	+0.06	Bandit-style allocation balances promising options with remaining uncertainty.
Late refinement	Top-tier local refinement	14	+0.03	The final phase extracts smaller gains near the best discovered candidates.
Final state	Adapted strategy sequence	–	score 0.743	Final score is 34.1% higher than the static baseline reported for the case study.

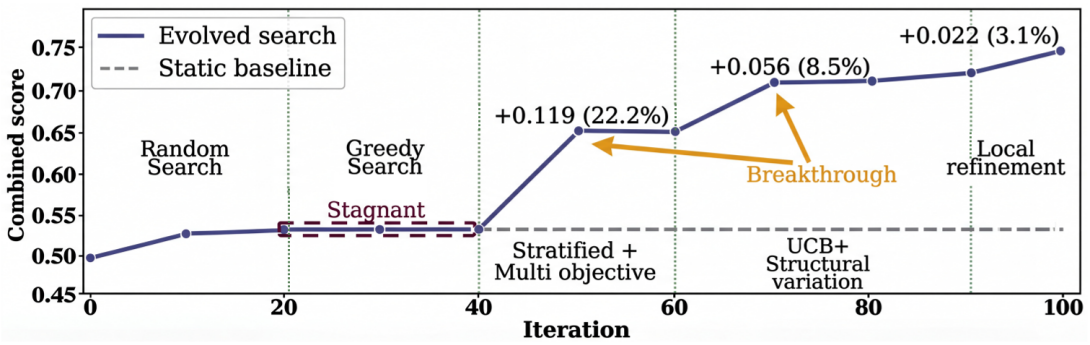


Figure 4.2: Strategy evolution on the signal-processing task: EvoX switches from uniform random sampling to greedy, stratified multi-objective, UCB, and top-tier refinement strategies [27].

4.3.5 Cost-Quality Behavior

The paper's extended experiments show that strategy adaptation can improve cost-quality trade-offs. On the Heilbronn triangle task, EvoX initialized from different strategies continues improving beyond fixed versions of those strategies. The cost analysis indicates that adaptive strategy can reach target scores at lower generation

cost than some baselines. This matters because an autonomous discovery system is only useful if its search cost is justified by the improvement.

The search-evolution analyses also reveal interpretable phases. For signal processing, EvoX moves from random sampling to greedy sampling, stratified multi-objective sampling, UCB selection, and top-tier refinement. For circle packing, the system shifts from broad heuristic discovery to structural variation that discovers constrained optimization mechanisms and then to local refinement. These case studies make the method more credible because they show how adaptation changes behavior, not merely that a final score is higher.

4.4 Discussion

4.4.1 Strengths

EvoX's first strength is that it targets a real weakness in fixed evolutionary search. Search policies are rarely universally optimal, and EvoX provides a concrete mechanism for changing them. Its second strength is that strategies are represented as executable code, which gives them more expressive power than scalar hyperparameters. Its third strength is evaluation breadth across mathematics, systems, and algorithmic tasks. Its fourth strength is interpretability of strategy trajectories: the system can report which strategy was used at each phase and how much improvement followed.

The method also has a pragmatic advantage. It does not require abandoning the evaluator-guided loop. Instead, it inserts an adaptive layer around a familiar population database. This makes the idea easier to integrate into existing evolutionary coding systems.

Another strength is that EvoX makes search behavior inspectable at the strategy level. Because strategies are executable objects with histories, the analysis can ask whether improvement came from exploration, exploitation, constraint handling, or local refinement. That is more informative than reporting only a final best score, especially when the goal is to understand how an autonomous discovery run changes over time [27].

4.4.2 Limitations

EvoX inherits the evaluator dependence of AlphaEvolve. It can adapt the search strategy only with respect to the measured score. If the evaluator is incomplete, strategy evolution may become a more efficient way to overfit. The method also adds complexity: there is now a strategy generator, a strategy validity test, a strategy history, and a progress threshold. These components introduce new hyperparameters and new failure modes.

Another limitation is that strategy generation itself depends on an LLM's ability to write valid and useful code. Invalid strategies must be rejected. Valid strategies may still encode short-sighted behavior. Finally, the reported results depend on powerful models and fixed budgets; the cost-benefit balance may differ for smaller models or very expensive evaluators.

EvoX also shifts some design burden from choosing a fixed strategy to defining the strategy interface. If the interface is too narrow, meta-evolution can only tune familiar behavior. If it is too broad, generated strategies become harder to validate and compare. The method therefore highlights a recurring systems trade-off: autonomy increases when interfaces are expressive, but reliability improves when they are constrained.

4.4.3 Position in the Review

EvoX occupies the middle point in the report's autonomy arc. It is more adaptive than AlphaEvolve because the search strategy changes over time, but it is less autonomous than CORAL because the retrieve-propose-evaluate-update loop remains externally organized. The system still has a central controller, a defined strategy interface, and a fixed trigger for strategy evolution.

This middle position is valuable. It shows that not all autonomy needs to be agent-level autonomy. Sometimes the most useful step is to identify a fixed design knob and make it adaptive. EvoX does exactly that for candidate selection and variation strategy.

4.5 Summary

EvoX demonstrates that the search strategy in LLM-driven evolution can itself be evolved. Its two-level loop provides a mechanism for escaping stagnation and adapting to different stages of a search run. The next chapter analyzes CORAL, which goes beyond strategy adaptation by giving autonomous agents control over more of the discovery process and by using shared persistent memory for coordination.

Chapter 5

CORAL: Autonomous Multi-Agent Evolution

5.1 Background and Motivation

CORAL begins with the claim that fixed evolutionary search leaves too many important decisions outside the agent. A fixed sampler decides what context to retrieve. A fixed loop decides when to evaluate. A fixed update rule decides what to store. For difficult open-ended problems, these are not merely implementation details; they are part of the search problem. An agent may need to inspect a failure log, run a local diagnostic, write a note about a bottleneck, or pivot away from a stale approach before the next official evaluation.

This motivation is especially strong in multi-agent settings. Many multi-agent systems use predefined roles and communication structures. CORAL instead emphasizes horizontal parallelism: several long-running agents explore in parallel and coordinate indirectly through shared persistent memory. This design avoids assuming that the best task decomposition is known in advance, and it connects to work on collaborative agent frameworks and memory-centered long-horizon agents [26, 6, 46, 51].

5.2 CORAL Method

5.2.1 Loop Formulation

CORAL formulates an open-ended discovery step as four stages: retrieve, propose, evaluate, and update. In fixed search, external rules implement most of these stages. In autonomous single-agent evolution, one agent chooses more of the actions. In autonomous multi-agent evolution, multiple agents each operate in private workspaces while reading and writing shared memory. The memory contains attempts, notes, and skills.

The paper defines an open-ended task by a task description x and an evaluator

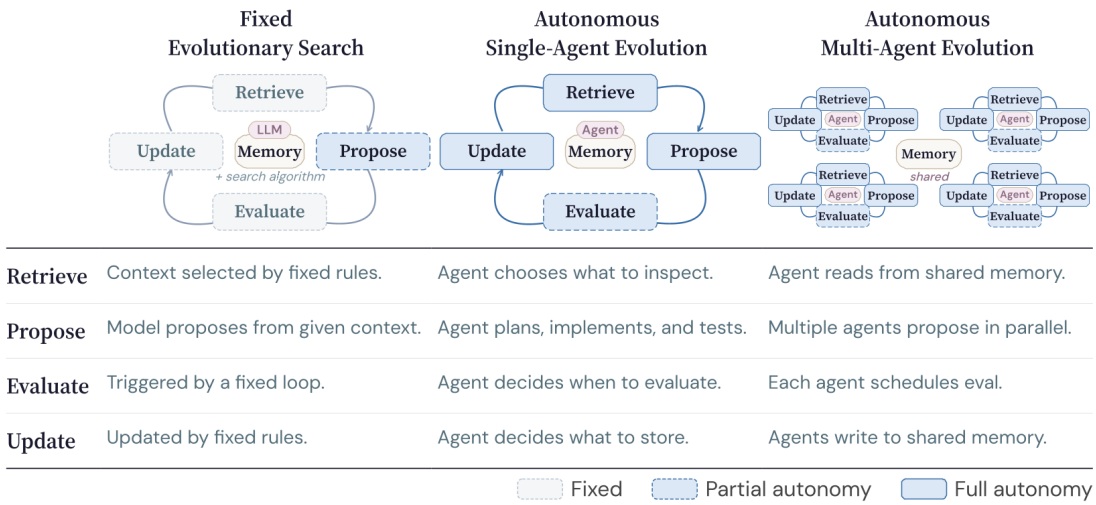


Figure 5.1: Three paradigms for LLM-based open-ended discovery: fixed search, autonomous single-agent evolution, and autonomous multi-agent evolution [38].

E. For a candidate solution y,

$$E(x, y) := (s, f), \tag{5.1}$$

where s is the scalar score and f is auxiliary feedback. Let M_t be shared persistent memory at step t. The CORAL loop is

$$\hat{M}_t = \text{Retrieve}(M_t), \tag{5.2}$$
$$y_{t+1} \sim \text{Propose}(\cdot \mid x, \hat{M}_t), \tag{5.3}$$
$$(s_{t+1}, f_{t+1}) = E(x, y_{t+1}), \tag{5.4}$$
$$M_{t+1} = \text{Update}(M_t, y_{t+1}, s_{t+1}, f_{t+1}). \tag{5.5}$$

The difference from fixed search is that the agent, rather than a hard-coded sampler, chooses much of the retrieval, proposal, local testing, and memory-writing behavior.

This formulation highlights the difference between LLM calls and agents. An LLM call generates from a prompt. An agent maintains an ongoing trajectory, interacts with files and tools, decides when to run tests, and can externalize knowledge for future use. CORAL's claim is that this persistent agency improves open-ended search because it allows knowledge to accumulate outside any single prompt window. That design is related to ReAct-style reasoning-action loops, Reflexion-style verbal feedback, and memory systems for persistent LLM agents [49, 41, 36].

5.2.2 Shared Memory

CORAL's feedback comes from task evaluators, local tests, shared attempts, notes, and heartbeat prompts. Attempts record evaluated solutions and scores. Notes capture observations, reflections, and domain insights. Skills package reusable pro-

cedures, often with a description and supporting files. This memory structure is richer than a flat program database because it separates evaluated artifacts from human-readable or agent-readable knowledge.

Memory curation matters because shared memory can become noisy. CORAL addresses this with heartbeat interventions. Reflection encourages agents to write structured notes. Consolidation turns repeated observations into reusable skills. Redirection prompts agents to pivot when progress stalls. These interventions preserve autonomy while nudging agents to maintain useful memory.

5.2.3 Heartbeat and Safeguards

The action protocol is implemented through an agent manager and command-line interface. Agents work in isolated git worktrees. They can inspect shared attempts, read notes, use skills, edit code, run local tests, and submit official evaluations. The manager monitors agent health and evaluation results. When a heartbeat action triggers, the manager interrupts and resumes the agent with a prompt containing the latest evaluation result and the relevant reflection, consolidation, or redirection instruction.

This protocol is more operationally complex than AlphaEvolve or EvoX, but it gives the agents a larger action space. The agent can decide that it needs more diagnosis before evaluation, or that a previous note suggests a better parent attempt. In systems terms, CORAL moves from an algorithm that calls an LLM to an infrastructure that hosts agents.

5.2.4 Multi-Agent Coordination

Each CORAL agent follows a loop that is intentionally not rigidly specified. It reads shared memory, plans an approach, edits code in its private worktree, runs local checks, submits evaluations, and writes discoveries back to memory. Other agents can then build on those attempts or notes. The coordination is indirect: agents do not need explicit messages or predefined roles. Shared artifacts become the communication medium.

5.3 Evaluation

5.3.1 Tasks and Results

CORAL is evaluated on mathematical optimization, systems optimization, Polyominoes, and kernel engineering. The paper reports new state-of-the-art results on several tasks and substantially higher improvement rates than fixed evolutionary search baselines. The headline result is Anthropic's kernel engineering task, where four co-evolving agents reduce the cycle count from a previous best of 1363 to 1103, a

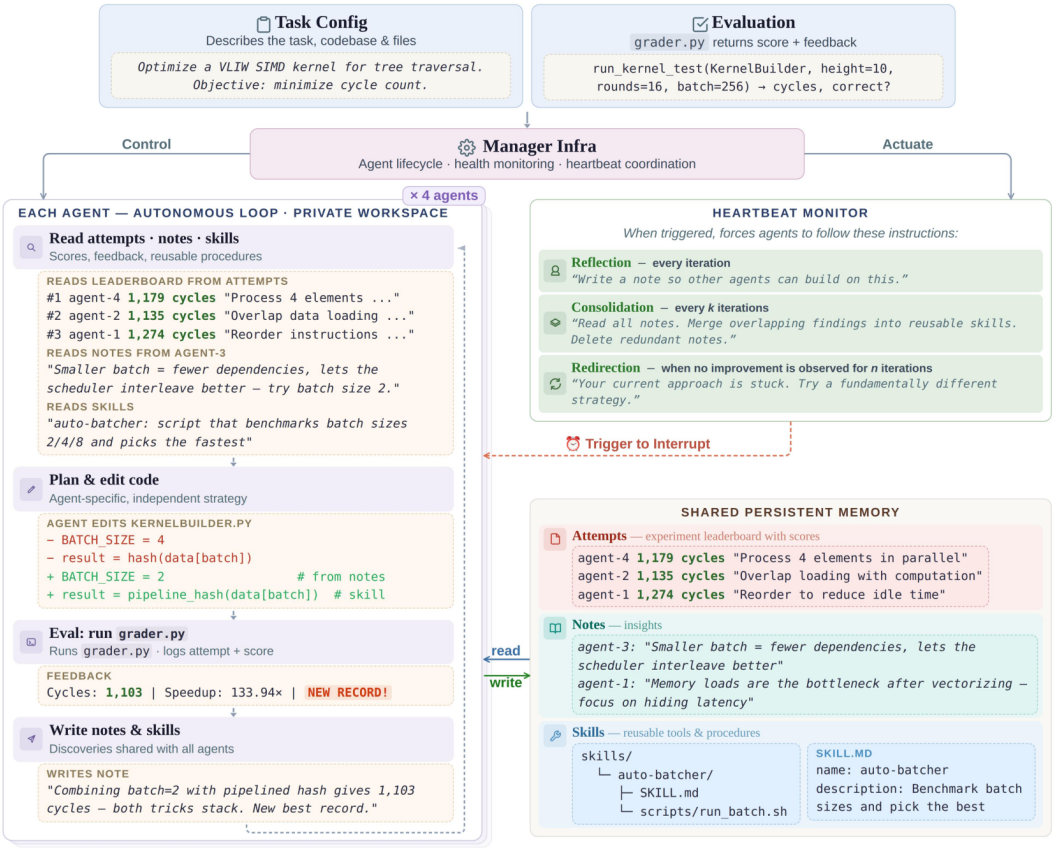


Figure 5.2: CORAL framework architecture with isolated worktrees, shared memory, and heartbeat monitoring [38].

reported 20 percent improvement over the prior best. Kernel-engineering and general software-agent benchmarks make this a relevant stress test for autonomous engineering, not only for mathematical search [35, 48, 44].

The broader task suite is important because it tests whether the framework is a general agent infrastructure rather than a kernel-specific tool. CORAL reports strong results across 11 mathematical and systems tasks, and the presentation slides emphasize that convergence often occurs in far fewer evaluations than fixed baselines.

Table 5.1: Selected CORAL multi-agent results reported in Table 2 of the paper.

Task	SOTA	1-agent	4-agent	Gain
Kernel Eng. (↓)	1363	1350	1103	18.30%
Polyominoes (↑)	87.0	80.2	84.2	4.99%
Circle-Pack. (↑)	2.6359	2.3531	2.5391	7.90%
MMD-14-3 (↓)	4.16	4.53	4.19	7.51%
Cloudcast (↓)	632.7	849.4	672.8	20.80%
LLM-SQL (↑)	0.730	0.693	0.730	5.34%

5.3.2 Baselines and Budgets

CORAL compares against fixed evolutionary search baselines including OpenEvolve, ShinkaEvolve, and EvoX, as well as single-agent CORAL and best-of-four independent single-agent runs in selected ablations. This baseline structure is important because a multi-agent system might improve simply by spending more compute. Comparing four co-evolving agents with the best of four independent runs helps test whether shared memory and co-evolution add value beyond parallelism.

The protocol uses wall-clock budgets and task-specific evaluation settings. For standard mathematical and systems tasks, methods are run under matched budgets. For stress tests, experiments stop under no-improvement or time conditions. The use of isolated workspaces and evaluator separation is part of the protocol, because without those safeguards agents could interfere with each other or inspect private grading logic.

5.3.3 Co-Evolution Versus Parallelism

The kernel engineering result is the clearest example of multi-agent advantage. A single CORAL agent already reaches a strong result quickly, but four co-evolving agents push the frontier further. The paper's analysis indicates that agents build on each other's commits and notes. This matters because the gain is not merely from running four independent searches; it comes from diffusion of discoveries through shared memory.

The standard task results show a complementary pattern. Single-agent CORAL often improves sample efficiency because agents decide when to evaluate and can run local tests before official submission. Multi-agent CORAL helps most when a single agent plateaus and when different strategies can be pursued in parallel. The result is a nuanced view: multi-agent organization is not automatically better for every task, but it is valuable when search diversity and knowledge transfer are limiting factors.

5.3.4 Efficiency and Cost

CORAL's efficiency should be interpreted carefully. The system may use fewer official evaluations because agents reason, inspect, and locally test before submitting. That is good when official evaluations are expensive or when failed submissions provide little value. But the reasoning time and LLM API cost still count. A fair deployment comparison must measure wall-clock time, model cost, evaluator cost, and human monitoring cost.

The framework's heartbeat and memory mechanisms also add overhead. The manager must track sessions, interrupts, worktrees, process health, and shared artifacts. This overhead is reasonable for high-value discovery tasks, but it may be excessive for small optimization problems where a fixed evolutionary loop is suffi-

Table 5.2: Selected CORAL ablation evidence for memory and co-evolution.

Setting	With mechanism	Without / baseline	Dir.	Interpretation
Knowledge accumulation, Kernel Eng.	1350 cycles	1601 cycles	↓	Removing memory hurts the single-agent run.
Knowledge accumulation, Polyominoes	80.2	77.3	↑	Notes and skills improve the final score.
Knowledge accumulation, Txn Scheduling	4566	4444	↑	Stored knowledge improves scheduling search.
Co-evolution, Kernel Eng.	1103 cycles	1180 cycles	↓	Shared multi-agent evolution beats independent runs.
Co-evolution, Polyominoes	84.2	80.8	↑	Shared memory improves over best independent run.
Co-evolution, Txn Scheduling	4694	4629	↑	Collaboration adds value beyond parallelism.

cient.

5.3.5 Task Interface

CORAL's generalization claim rests on its task interface. A task is specified by a configuration file, seed repository, and grader. Agents do not need a hand-written role decomposition for each task. Instead, they receive the task description and use shared memory to organize their search. This makes CORAL more task-agnostic than a specialized multi-agent workflow.

However, the generality still depends on the availability of a reliable grader and a workspace where agents can safely act. Tasks that require external physical experiments or manual judgment are harder. Tasks with hidden constraints require careful evaluator design. Generalization is therefore an infrastructure property as much as an algorithmic property.

5.3.6 Knowledge Transfer

CORAL's behavioral analyses are one of its strongest contributions. The paper reports statistics on local testing, knowledge creation, knowledge access, and read-to-improvement rates. It also analyzes cross-agent transfer, showing that attempts using another agent's commit can have higher improvement rates in some tasks.

These analyses help explain why the system works.

The reported trajectory statistics make the point more concrete. Across standard tasks, CORAL reports a 24 percent improvement rate, local testing on 24 percent of attempts, a 37 percent test-to-improvement rate, 0.05 knowledge artifacts per attempt, and a 26 percent read-to-improvement rate. On the more knowledge-intensive Polyominoes task, the improvement rate is 30 percent and the system records 0.55 knowledge artifacts per attempt. On kernel engineering, local testing rises to 57 percent, knowledge artifacts to 0.68 per attempt, and read-to-improvement to 55 percent. These numbers support the paper's argument that memory and local diagnosis are active parts of the search process, not merely logging features.

The qualitative examples of notes and skills are also important. A note about a kernel bottleneck can capture why one transformation works and why another fails. A skill can package a reusable method for future attempts. This kind of artifact is closer to how human engineering teams accumulate knowledge than a simple list of scored programs.

Table 5.3: Selected CORAL cross-agent transfer evidence.

Task / analysis	Transfer statistic	Improvement statistic	Interpretation
Kernel engineering	36% of attempts use another agent's commit	Transfer attempts improve at 17% versus 9% overall	Cross-agent parent use is associated with higher-yield attempts.
Polyominoes	12% direct transfer	Transfer attempts improve at 50% versus 19% overall	Transfer is rarer but highly productive when it occurs.
Kernel strategy diversity	Average pairwise strategy overlap 0.43	All four agents reach the best 1103-cycle result	Agents share useful discoveries while retaining partly distinct search behavior.
Polyomino strategy diversity	Average pairwise strategy overlap 0.31	More than half of each agent's strategy vocabulary is unique	Shared memory does not collapse all agents into identical trajectories.
Kernel workload balance	Each agent produces roughly 130–165 attempts	Each agent obtains 10–16 improvements	The best result is not explained by only one active worker while others idle.

5.4 Discussion

5.4.1 Strengths

CORAL's first strength is that it delegates more of the search process to agents without removing safeguards. Agents can choose actions, but evaluator isolation, worktree separation, and manager monitoring preserve control. Its second strength is shared persistent memory. Attempts, notes, and skills support cross-agent knowledge transfer. Its third strength is trajectory analysis, which provides evidence about how agents use memory and why multi-agent co-evolution helps.

The fourth strength is the framework orientation. CORAL is not just a new sampler; it is an operating environment for autonomous discovery. That makes it relevant to software engineering practice, where process management, session recovery, and reproducibility are as important as the generation algorithm.

This framework orientation also makes CORAL useful for tasks where progress requires diagnosis before mutation. Agents can inspect failures, formulate local hypotheses, run supporting checks, and externalize observations into notes or skills. The resulting memory is richer than a ranked list of candidates because it records why an attempt may matter, not only whether it improved the score [38].

CORAL's design is also valuable because it separates coordination from direct communication. Agents do not need a scripted debate protocol or fixed specialist roles; they coordinate by leaving evaluated artifacts and reusable knowledge in a common store. This makes the method closer to collaborative engineering practice, where durable work products often matter more than transient messages.

5.4.2 Limitations

The main limitation is complexity. CORAL requires an agent manager, isolated workspaces, shared memory, heartbeat configuration, process control, and grader isolation. This is much more machinery than a fixed evolutionary loop. The second limitation is cost. Multi-agent runs can multiply model usage, and agents may spend many tokens reasoning or inspecting files before evaluation. The third limitation is memory quality. Shared notes and skills can help, but they can also become stale, redundant, or misleading.

Another limitation is that agent autonomy can make trajectories harder to reproduce. Long-running agents interact with tools, histories, and timing. Two runs with the same seed may diverge substantially. This is acceptable for exploratory discovery, but it complicates scientific comparison and production certification.

The memory layer introduces a related governance problem. Shared notes and skills can accelerate transfer, but they also require policies for deduplication, aging, attribution, and removal. Without such policies, the shared workspace can accumulate misleading generalizations or cause agents to over-exploit a popular but

exhausted direction.

5.4.3 Related Literature

CORAL connects to autonomous coding agents, self-reflection, memory consolidation, and multi-agent collaboration. Its distinctive feature is that these ideas are embedded in an evaluator-guided optimization loop. The agents are not merely completing a software issue; they are repeatedly searching for better solutions under a quantitative objective [30, 41, 46].

The framework also differs from role-based multi-agent systems. Rather than assigning one agent to be a planner, another to be a coder, and another to be a reviewer, CORAL starts agents symmetrically. Roles and strategies can emerge through interaction with shared memory. This horizontal design is better aligned with open-ended search, where the best division of labor may not be known beforehand.

5.4.4 Position in the Review

CORAL is the third step in the report's progression. AlphaEvolve fixes the search scaffold and evolves programs. EvoX evolves both programs and the search strategy. CORAL delegates the loop to agents and supports coordination through shared memory. The degree of autonomy increases at each step.

This does not mean CORAL strictly dominates the earlier systems. AlphaEvolve may be preferable when the task is well formalized and high-throughput evaluation is enough. EvoX may be preferable when the main weakness is search-policy stagnation. CORAL becomes most attractive when progress depends on long-horizon reasoning, local diagnosis, knowledge externalization, and diverse parallel exploration.

5.4.5 Comparison with AlphaEvolve and EvoX

The comparison shows that the papers are not simple variants of one method. They move the boundary between fixed infrastructure and agent-controlled behavior. AlphaEvolve asks how to make LLM proposals reliable. EvoX asks how to make the search policy adaptive. CORAL asks how to make the entire search process more autonomous while retaining operational safeguards.

5.4.6 Design Trade-Offs

For a practitioner, the choice among these designs should be workload-driven. If the evaluator is fast and the task has a clear code representation, an AlphaEvolve-style system may be sufficient. If the search repeatedly stagnates because different phases require different exploration strategies, EvoX-style meta-evolution is attractive. If

Table 5.4: Compact comparison of the three focal papers.

Paper	Core idea	Memory structure	Main autonomy level
AlphaEvolve	LLM-generated code diffs selected by evaluation	Program database with scores and feedback	LLM as mutation operator in fixed loop
EvoX	Co-evolve solutions and search strategies	Solution database plus strategy history	Adaptive strategy inside controlled loop
CORAL	Autonomous agents coordinate through shared memory	Attempts, notes, and skills in persistent memory	Agents control retrieve, propose, evaluate, update behavior

progress requires investigation, local testing, memory consolidation, and parallel exploration, CORAL-style autonomous agents may justify their cost.

The safest deployment pattern is likely hybrid. Fixed infrastructure should enforce evaluator isolation, budget limits, logging, and rollback. Adaptive strategies should decide how to sample and vary candidates. Agents should be given autonomy where diagnosis and knowledge accumulation matter, but not where safety requires deterministic control.

5.5 Summary

CORAL reframes open-ended discovery as autonomous multi-agent evolution. Its key contribution is not only higher reported scores, but a richer architecture for shared memory, agent lifecycle management, and heartbeat-driven reflection. It shows the potential of horizontal multi-agent search while making clear that autonomy introduces new costs and governance problems.

Chapter 6

Cross-Paper Synthesis and Open Challenges

6.1 Autonomy Progression

The three papers can be summarized along three axes: autonomy, search adaptivity, and knowledge accumulation. AlphaEvolve has strong automated evaluation and program memory, but a mostly fixed search scaffold. EvoX adds search adaptivity by evolving the strategy that selects parents and variation operators. CORAL increases autonomy and enriches knowledge accumulation through shared persistent memory across agents.

These axes are connected. More autonomy requires better memory because agents need a stable substrate for coordination. More search adaptivity requires better feedback because the system must know when a strategy is failing. More knowledge accumulation requires better governance because memory can amplify both insights and mistakes.

6.2 Evaluation and Measurement

All three systems avoid large human-labeled datasets by using evaluator feedback. This is one of their main advantages. The human burden shifts from labeling examples to designing tasks, writing evaluators, providing seed code, and validating results. This shift is powerful because a good evaluator can produce many training-like signals during search. It is also risky because evaluator design becomes the bottleneck.

AlphaEvolve uses evaluator scores directly to select programs. EvoX uses score progress to select strategies. CORAL uses scores and feedback as part of a broader memory and reflection process. In each case, the system can only optimize what the feedback exposes. For scientific and engineering discovery, the quality of task formalization is therefore as important as the quality of the LLM.

6.2.1 Benchmark Assumptions

The papers' results depend on hidden assumptions about budgets, models, and evaluators. A method using a stronger model may look better than a method using a weaker model. A method with more parallel agents may look better in wall-clock time while costing more in tokens. A method evaluated on a task with a forgiving grader may find high scores that do not generalize.

Future evaluation should report several resource axes together: final score, number of official evaluations, number of local tests, wall-clock time, LLM token cost, evaluator cost, and human intervention. It should also include evaluator audits. CORAL's documentation of evaluator bug fixes is a useful example; such audits should become standard.

6.3 System Design Trade-Offs

The three systems match different workloads. AlphaEvolve is well suited for high-value optimization tasks where the editable code boundary and evaluator are clear. EvoX is well suited for tasks where search behavior must change over time. CORAL is well suited for tasks where progress requires accumulated qualitative insight, multiple lines of attack, and long-running agent trajectories.

Table 6.1: Workload fit across the three systems.

System	Best fit	Main caution
AlphaEvolve	Clear evaluator, code-represented candidates, high-throughput search	Fixed strategy may stagnate; task setup remains human-intensive
EvoX	Search landscapes with phase changes and strategy-sensitive progress	Added complexity in strategy generation and validation
CORAL	Open-ended tasks needing diagnosis, memory, and parallel exploration	Higher operational cost and harder reproducibility

6.4 Operating-Systems Perspective

The operating-systems relevance of these papers is not only that some reported tasks are systems tasks. The deeper connection is that autonomous discovery systems themselves are managed execution environments. They schedule work, isolate processes, control access to state, maintain logs, recover from failures, and expose interfaces between trusted and untrusted components. In that sense, AlphaEvolve, EvoX,

and CORAL should be read not only as machine-learning methods, but also as early examples of operating-system design for LLM-driven experimentation. This view is consistent with recent arguments that AI systems are beginning to reshape systems research itself [8].

AlphaEvolve presents the most controlled version of this idea. The candidate program is confined to evolve blocks, the evaluator is external, and the program database mediates which artifacts can influence future prompts. This resembles a narrow service boundary: the system permits variation inside a specified region while keeping the evaluator, benchmark harness, and deployment decision outside the generated code. Such a boundary is essential for infrastructure work. If an optimization agent can edit the grader, change the benchmark workload, or bypass a correctness check, the resulting score is no longer evidence of improvement. AlphaEvolve's strongest production examples are credible because the editable surface and validation path are constrained.

EvoX shifts the systems question from isolation alone to control-policy adaptation. A traditional scheduler uses fixed rules or hand-tuned policies; EvoX suggests that the policy guiding search can itself be revised when the workload state changes. The analogy is useful but should not be overstated. EvoX is not an operating-system scheduler in the ordinary sense. Still, it addresses a familiar systems problem: a static policy can be reasonable on average and poor at a particular phase of a run. The strategy history, windowed progress score, and stagnation trigger act like a management layer that monitors whether the current policy is still productive. This is the central technical bridge between meta-evolution and systems design: adaptivity must be tied to measurable progress, not to the model's confidence in its own plan.

CORAL makes the operating-system analogy explicit. Agents run in isolated worktrees, interact through a shared hub, submit official evaluations, and receive heartbeat interventions from a manager. The framework has to solve resource-management and coordination problems that do not appear in a single LLM call. Agents may stall, duplicate work, write low-quality notes, or pursue stale leads. The manager therefore becomes part of the method: it maintains process health, controls access to official evaluation, and injects reflection or redirection when needed. This is why CORAL's contribution is not just "more agents." Its contribution is an execution substrate that lets agents behave autonomously while keeping their activity observable and partly governable.

This perspective also clarifies why evaluator-guided discovery is different from ordinary benchmark automation. In benchmark automation, the system repeatedly measures a fixed artifact. In autonomous discovery, the system also creates the artifact, changes its own context, and may generate durable memory that affects future attempts. The risk surface is therefore larger. A flawed benchmark does not merely produce one bad measurement; it can shape the search trajectory, select misleading examples into memory, and bias later generations. The more autonomous

the system becomes, the more important it is to treat evaluation, memory, and artifact provenance as part of the trusted computing base.

The three papers imply a layered architecture for future engineering systems. The bottom layer should be a sandboxed execution environment with resource limits, deterministic environment capture where possible, and separation between editable candidate code and private evaluator code. The next layer should be an evaluation service that records official submissions, local tests, grader versions, and score metadata. Above that, a memory service should store attempts, summaries, notes, and reusable skills with provenance rather than treating prompt context as an informal scratchpad. The policy layer can then choose parents, strategies, agents, and budgets. Finally, a human-facing review layer should expose the exact diff, supporting evidence, cost, and rollback path for any candidate that is accepted beyond the search environment.

This layered view improves the narrative connection among the papers. AlphaEvolve emphasizes the candidate and evaluation layers: code changes become credible when they are scored by a trusted evaluator and stored in a structured database. EvoX emphasizes the policy layer: the search controller should respond to stagnation and population state. CORAL emphasizes the memory and process layers: long-running agents need shared state, heartbeat management, and workspace isolation. The systems are therefore complementary rather than mutually exclusive. A practical autonomous engineering platform could use AlphaEvolve-style code boundaries, EvoX-style adaptive strategy selection, and CORAL-style shared memory under one managed execution service.

This synthesis also prevents an overly simple conclusion that more autonomy is always better. More autonomy increases the number of useful decisions the system can make, but it also increases the number of ways the system can become difficult to audit. A fixed evolutionary loop may be less creative, but it is easier to reproduce and bound. A multi-agent system may discover more diverse solutions, but it can also obscure which decision caused a breakthrough or a failure. For operating-systems and infrastructure applications, the right target is not maximal autonomy. The right target is controlled autonomy: agents should have freedom where exploration, diagnosis, and memory writing create value, while fixed infrastructure should own safety-critical boundaries such as grader isolation, budget enforcement, permission control, and deployment approval.

6.5 Open Challenges

The first open problem is evaluator robustness. Agentic systems will exploit whatever signal they receive. Strong grader isolation, adversarial evaluator tests, and post-hoc verification are necessary. The second problem is memory management. Program databases, strategy histories, notes, and skills can become large and noisy. Future

systems need retrieval mechanisms that preserve useful diversity without overwhelming the prompt.

The third problem is budget governance. Autonomous systems need explicit policies for when to spend more model calls, when to run expensive evaluations, and when to stop. The fourth problem is reproducibility. Long-running agent systems should produce audit trails that make it possible to reconstruct why a candidate was found. The fifth problem is human oversight. For production infrastructure, discovered code must be inspectable, testable, and rollback-safe.

These problems are connected rather than independent. A flawed evaluator can turn memory quality into a liability, because agents may preserve and reuse evidence from a bad signal. Weak budget control can make benchmark fairness impossible, because a method may spend more hidden compute than its baselines. Poor reproducibility makes safety review harder, because engineers cannot reconstruct the path from seed program to final candidate. The open challenge is therefore to design discovery systems in which evaluation, memory, budgeting, and review are treated as one control problem.

6.6 Design Guidelines for Future Systems

Interface-first autonomy. The comparison suggests five practical directions for future autonomous discovery systems. They all point to the same architectural principle: autonomy should be expanded through explicit interfaces, not through unrestricted access to the whole experiment. A strong system needs an evaluator interface, a candidate boundary, an evidence-bearing memory store, an accountable search policy, and a human-readable acceptance record.

Evaluator as contract. The evaluator should be treated as a contract. In all three papers, evaluation defines what progress means: AlphaEvolve depends on exact or production-grade verification, EvoX depends on measuring strategy progress, and CORAL depends on local tests and official graders. Future systems should expose candidates only to the information needed for search, while hiding private tests, benchmark logic, and acceptance criteria that generated code could manipulate. The evaluator should also separate validity from quality. A useful report does not return only one scalar; it records hard correctness checks, objective performance, diagnostic feedback, and evaluator version metadata. This prevents a candidate from looking successful merely because it is fast, brittle, or exploiting an artifact of the benchmark.

Candidate boundary. The editable candidate boundary should be narrow enough to protect the experiment and broad enough to allow real discovery. AlphaEvolve's evolve-block interface and CORAL's isolated worktrees illustrate two versions of this idea. For operating-systems and infrastructure tasks, the boundary must specify editable files, allowed side effects, and forbidden shortcuts such as disabling checks, changing timeouts, or caching hidden answers. Larger edits are

sometimes necessary, but they require stronger provenance: the final record should include the parent candidate, the diff, the local tests, the official evaluator version, and any memory items that influenced the change.

Memory as evidence. Memory should be treated as evidence rather than as an unstructured transcript. AlphaEvolve's program database, EvoX's strategy history, and CORAL's shared notes and skills all shape future search. A reliable memory item should therefore carry provenance, confidence, and conditions of use. Attempts should record scores, validity, parents, diffs, and evaluator versions. Notes should point to the observations that support them. Skills should record where they helped and when they may no longer apply. Memory also needs a lifecycle: stale, duplicated, or contradicted items should be summarized, retired, or marked obsolete rather than silently reused.

Auditable policy and budget. Adaptive policy should be auditable. EvoX shows the benefit of changing the search strategy when progress stalls; CORAL shows the benefit of giving agents more freedom over long horizons; AlphaEvolve shows the simplicity of a fixed scaffold. A mature system should combine these strengths while reporting the true cost of search. Official evaluator calls are not enough. Reports should include local tests, failed executions, model calls, token usage, wall-clock time, parallelism, and human interventions. Stopping rules should be similarly explicit: the system should explain whether it continued, changed strategy, requested human review, or terminated because of stagnation, uncertainty, or diminishing returns.

Human-readable acceptance. Human review should sit at the boundary between discovery and acceptance. The goal of these systems is to reduce manual search, not to remove responsibility for final claims. A scientific result should end with the statement of the improvement, the artifact, the verifier, and an independent reproduction path. An engineering result should end with the diff, benchmark settings, regression tests, resource cost, expected impact, and rollback plan. This acceptance package is what converts an autonomous search result into something that can be trusted, maintained, and cited. Autonomy should expand the search frontier; it should not obscure the evidence needed to accept the result.

6.7 Summary

The synthesis supports a balanced conclusion. The field is moving toward more autonomous systems, but autonomy is valuable only when paired with reliable evaluation, useful memory, and operational safeguards. AlphaEvolve, EvoX, and CORAL each solve one part of the problem. The next generation of systems will likely combine their strengths.

Chapter 7

Conclusion and Future Work

This report reviewed AlphaEvolve, EvoX, and CORAL as three stages in the development of autonomous agent systems for discovery and engineering. AlphaEvolve shows that frontier LLMs can become powerful discovery engines when embedded in an evaluator-guided evolutionary loop. EvoX shows that the search strategy itself should adapt to the state of the population and the stage of the run. CORAL shows that long-running agents with shared persistent memory can coordinate, transfer discoveries, and push difficult open-ended optimization tasks beyond fixed baselines.

The integrated thesis is that progress comes from moving the right decisions into the agentic layer while keeping the right safeguards in fixed infrastructure. LLMs should be allowed to propose code, diagnose failures, adapt strategies, and write reusable knowledge. Infrastructure should enforce evaluator isolation, budget limits, logging, reproducibility, and final validation. The strongest future systems will not be fully fixed loops or uncontrolled autonomous agents; they will be carefully engineered hybrids.

This conclusion is deliberately more architectural than model-centric. The three papers use strong language models, and model quality clearly matters, but the most transferable ideas are not only about which backbone generates code. They are about how the system converts uncertain generations into reliable evidence. AlphaEvolve does this through structured diffs, execution feedback, and a program database. EvoX does it by measuring whether a strategy actually improves the population over a window. CORAL does it by turning long-running agent activity into shared attempts, notes, skills, and official evaluations. In each case, the language model is powerful because it is embedded in a process that can reject, remember, and revise.

The review also shows why autonomous discovery should be evaluated as a workflow rather than as a single algorithm. A final score is important, but it is not enough to judge whether the system is useful. The reader should ask how the task was formalized, which parts of the workspace were editable, how evaluator leakage was prevented, how much compute was spent, how memory was curated, and how the final artifact can be inspected. These questions are sometimes treated as im-

plementation details, but they determine whether a reported discovery can survive outside the experimental run. For operating-systems and engineering applications, this distinction is central: a fast but unauditable optimization is not a deliverable system improvement.

The practical lesson is therefore conditional. If a task has a reliable evaluator, a bounded candidate representation, and a high payoff for small improvements, an AlphaEvolve-style loop is a strong starting point. If the run repeatedly stalls because different stages need different search behavior, EvoX-style strategy evolution is the natural extension. If the problem requires diagnosis, parallel exploration, qualitative memory, and long-running tool use, CORAL-style autonomous agents become more compelling. These choices should be made from the task structure, not from a general preference for more autonomy.

Future work should focus on five directions. First, evaluator design needs more formal auditing and adversarial testing. Second, memory systems need better mechanisms for deduplication, confidence, provenance, and retrieval. Third, budget-aware control should become a first-class part of agentic discovery. Fourth, multi-agent systems need clearer methods for measuring when collaboration adds value beyond parallel compute. Fifth, production deployment requires interfaces that let humans inspect, approve, and rollback discovered changes. If these challenges are addressed, autonomous agent systems could become a practical layer for accelerating algorithmic discovery and engineering optimization across many domains.

Bibliography

[1] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.

[2] Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W. Hoffman, David Pfau, Tom Schaul, Brendan Shillingford, and Nando de Freitas. Learning to learn by gradient descent by gradient descent. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[3] Wolfgang Banzhaf, Peter Nordin, Robert E. Keller, and Frank D. Francone. *Genetic Programming: An Introduction on the Automatic Evolution of Computer Programs and Its Applications*. Morgan Kaufmann, 1998.

[4] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023.

[5] Tianlong Chen, Xiaohan Chen, Wuyang Chen, Howard Heaton, Jialin Liu, Zhangyang Wang, and Wotao Yin. Learning to optimize: A primer and a benchmark. *Journal of Machine Learning Research*, 23(189):1–59, 2022.

[6] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, et al. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. *arXiv preprint arXiv:2308.10848*, 2024.

[7] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, et al. Symbolic discovery of optimization algorithms. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[8] Audrey Cheng, Shu Liu, Melissa Pan, Zhifei Li, Bowen Wang, Alex Krentsel, Tian Xia, Mert Cemri, Jongseok Park, Shuo Yang, et al. Barbarians at the gate: How AI is upending systems research. *arXiv preprint arXiv:2510.06189*, 2025.

[9] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[10] Alex Davies, Petar Veličković, Lars Buesing, Sam Blackwell, Daniel Zheng, Nenad Tomašev, Richard Tanburn, Peter Battaglia, Charles Blundell, András Juhász, et al. Advancing mathematics by guiding human intuition with AI. *Nature*, 600(7887):70–74, 2021.

[11] Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Francisco J. R. Ruiz, Julian Schrittwieser, Grzegorz Swirszcz, et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022.

[12] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. PromptBreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.

[13] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Anil Palepu, Petar Sirkovic, Artiom Myaskovsky, Felix Weissenberger, Keran Rong, Ryutaro Tanno, et al. Towards an AI co-scientist. *arXiv preprint arXiv:2502.18864*, 2025.

[14] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. EvoPrompt: Connecting LLMs with evolutionary algorithms yields powerful prompt optimizers. *arXiv preprint arXiv:2309.08532*, 2023.

[15] Erik Hemberg, Stephen Moskal, and Una-May O'Reilly. Evolving code with a large language model. *Genetic Programming and Evolvable Machines*, 25(2):21, 2024.

[16] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2024.

[17] John E. Hopcroft and Leslie R. Kerr. On minimizing the number of multiplications necessary for matrix multiplication. *SIAM Journal on Applied Mathematics*, 20(1):30–36, 1971.

[18] Yuki Imajuku, Kohki Horie, Yoichi Iwata, Kensho Aoki, Naohiro Takahashi, and Takuya Akiba. ALE-Bench: A benchmark for long-horizon objective-driven algorithm engineering. *arXiv preprint arXiv:2506.09050*, 2025.

[19] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021.

[20] Manuel Kauers and Jakob Moosbauer. Flip graphs for matrix multiplication. In *Proc. International Symposium on Symbolic and Algebraic Computation (ISSAC)*, pages 381–388, 2023.

[21] John R. Koza. Genetic programming as a means for programming computers by natural selection. *Statistics and Computing*, 4(2):87–112, 1994.

[22] Julian D. Laderman. A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications. *Bulletin of the American Mathematical Society*, 82(1):126–128, 1976.

[23] William B. Langdon and Riccardo Poli. *Foundations of Genetic Programming*. Springer, 2013.

[24] Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. ShinkaEvolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349*, 2025.

[25] Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O. Stanley. Evolution through large models. In *Handbook of Evolutionary Machine Learning*, pages 331–366. Springer, 2024.

[26] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for "mind" exploration of large language model society. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[27] Shu Liu, Shubham Agarwal, Monishwaran Maheswaran, Mert Cemri, Zhifei Li, Qiuyang Mang, Ashwin Naren, Ethan Boneh, Audrey Cheng, Melissa Z. Pan, et al. EvoX: Meta-evolution for automated discovery. *arXiv preprint arXiv:2602.23413*, 2026.

[28] Shu Liu, Mert Cemri, Shubham Agarwal, Alexander Krentsel, Ashwin Naren, Qiuyang Mang, Zhifei Li, Akshat Gupta, Monishwaran Maheswaran, Audrey Cheng, et al. SkyDiscover: A flexible framework for AI-driven scientific and algorithmic discovery, 2026. https://skydiscover-ai.github.io/blog.html.

[29] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

[30] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-Refine: Iterative refinement with self-feedback. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[31] Qiuyang Mang, Wenhao Chai, Zhifei Li, Huanzhi Mao, Shang Zhou, Alexander Du, Hanchen Li, Shu Liu, Edwin Chen, Yichuan Wang, et al. FrontierCS: Evolving challenges for evolving intelligence. *arXiv preprint arXiv:2512.15699*, 2025.

[32] Luke Metz, Niru Maheswaranathan, Jeremy Nixon, Daniel Freeman, and Jascha Sohl-Dickstein. Understanding and correcting pathologies in the training of learned optimizers. In *Proc. International Conference on Machine Learning (ICML)*, 2019.

[33] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.

[34] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

[35] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels? In *Proc. International Conference on Machine Learning (ICML)*, 2025.

[36] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

[37] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. ChatDev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924*, 2024.

[38] Ao Qu, Han Zheng, Zijian Zhou, Yining Yan, Yuhan Tang, Sarah Y. Ong, Fangzhou Hong, Kang Zhou, Chonghe Jiang, Minwei Kong, et al. CORAL: Towards autonomous multi-agent evolution for open-ended discovery. *arXiv preprint arXiv:2604.01658*, 2026.

[39] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.

[40] Asankhaya Sharma. OpenEvolve: An open-source evolutionary coding agent. *GitHub repository*, 2025. https://github.com/codelion/openevolve.

[41] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[42] Volker Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, 1969.

[43] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proc. European Conference on Computer Systems (EuroSys)*, 2015.

[44] Xingyao Wang et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

[45] Yiping Wang, Shao-Rong Su, Zhiyuan Zeng, Eva Xu, Liliang Ren, Xinyu Yang, Zeyi Huang, Xuehai He, Luyao Ma, Baolin Peng, et al. ThetaEvolve: Test-time learning on open problems. *arXiv preprint arXiv:2511.23473*, 2025.

[46] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Proc. Conference on Language Modeling (COLM)*, 2024.

[47] Minghao Yan, Bo Peng, Benjamin Coleman, Ziqi Chen, Zhouhang Xie, Zhankui He, Noveen Sachdeva, Isabella Ye, Weili Wang, Chi Wang, et al. PACEvolve: Enabling long-horizon progress-aware consistent evolution. *arXiv preprint arXiv:2601.10657*, 2026.

[48] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieber, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[49] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[50] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. ReEvo: Large language models as hyperheuristics with reflective evolution. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[51] Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Bryan Kian Hsiang Low, and Paul Pu Liang. MEM1: Learning to synergize memory and reasoning for efficient long-horizon agents. In *Proc. International Conference on Learning Representations (ICLR)*, 2026.